Evaluating Quantum Contextuality as an Error Mitigation Resource in the Mermin–Peres Magic Square Game

Varun Bhadurgatte Nagaraj

Round Rock High School, Round Rock, Tx

AP Research

Carabeth Daly

4/17/2026

5730

Abstract

The purpose of this study was to analyze the benefit of the preservation of quantum contextuality on Mermin-Peres magic square algorithm accuracy for noisy quantum hardware. To assess the benefit of preserving contextuality, four circuit configurations on qubit superconducting processors at IBM were tested: a non-contextual baseline, a circuit design preserving contextuality, one designed to fully preserve it, and another designed to partially preserve it. Each configuration was executed on nine query positions, with a total of 8,192 measurement shots. Fidelity and trace distance were used to evaluate the accuracy of the algorithm and the measurements of contextuality were based on violations of a non-contextuality inequality. All conditions used zero-noise extrapolation (ZNE) as an error mitigation method. The results from the experiments showed a general trend whereby the circuits designed to preserve contextuality outperformed the baseline in fidelity and trace distance. The partial preservation of contextuality yielded the best results overall. However, although the differences in the results were not statistically significant ($p = 0.144$ and $p = 0.164$) there was a large effect size ($\eta^2 \approx 0.178$) indicating a potentially meaningful underlying relationship between the two factors. These results provide preliminary evidence suggesting contextuality may function as an independent computational resource within NISQ devices and further support future circuit design.

Keywords: quantum contextuality, Mermin–Peres magic square, NISQ, error mitigation, zero-noise extrapolation, fidelity, superconducting qubits, quantum error mitigation, quantum resource theory, quantum circuits

Table of Contents

Introduction

Background

Quantum computers are widely known as the future of computing, yet there is a critical limitation (Feynman, 1982). The current generations of quantum computers (Noisy Intermediate-Scale Quantum/NISQ) are unreliable due to noise caused by limited qubit counts, gate errors, and measurement issues that degrade any algorithm run on these devices (Khollam, 2025). There are two main ways to reduce error: increase qubit counts or build noise-resistant architectures (Preskill, 2018). Since it is incredibly hard to scale qubits, mitigating these issues through noise-aware algorithms remains the most practical option (Government of Japan, 2024).

Contextuality is a fundamental property of quantum mechanics, describing how measurement outputs can be dependent on the compatible measurements done alongside it (Roca, 2019). In classical systems, objects can be assigned a value prior to observation since it is assumed objects exist independently of their measurement context. In contrast, a quantum system's qubits don't possess a definite value before measurement; rather, its value is fundamentally dependent on the mathematical context of its measurement (VDE, 2025). Researchers have identified contextuality as a key resource in improving error rates in future NISQ devices as the first step towards universal quantum calculation (Haug & Kim, 2023).

Purpose of Study and Research Question

The purpose of this study is to determine how contextuality influences the Mermin–Peres magic square algorithm functions on noisy quantum computers. The magic square is a well-known illustration of contextuality because even though the magic square provides many observable outcomes from the operators measured, no classical, non-contextual, theory can predict these outcomes (Brassard et al., 2005). This study aims to answer the question: "To what

extent does preserving contextuality improve the accuracy of the Mermin–Peres magic square algorithm on noisy quantum hardware?"

Key Definitions and Terminology

The term "contextuality" refers to the idea that the outcome of a measurement cannot be predetermined and instead depends on the specific set of measurements performed together.

"Magic" resources refer to non-stabilizer quantum resources used in computations that are not standard binary (Haug & Kim, 2023).

The measure of how close the actual result of the computation is to what should have theoretically happened is known as the "fidelity" of the calculation. The calculation of the fidelity is almost like that of the "trace distance," which provides a numerical measure of how similar two probability distributions are.

NISQ devices are quantum computers that do not possess the complete set of resources needed to implement error-correction schemes, nor do they have resistance to environmental noise or decay; therefore, they are referred to as "Noisy Intermediate-Scale Quantum" (NISQ) devices (Ivezic, 2018).

In quantum computing, "error mitigation" is a technique for reducing the effects of noise on quantum circuits. One technique for error mitigation is zero-noise extrapolation (ZNE), which allows for the correlation of an experimental result to a theoretical result (if there is one) from measurements on computations run at several different levels of noise.

Contextuality as a Computational Resource

According to Howard et al.'s (2014) interpretation, it is contextuality that gives rise to the "magic" that is required for universal quantum computation. Furthermore, contextuality contributes non-classical correlation that makes computational speed up possible, meaning that

although there are no classical correlations to provide quantum speed-up, entanglement does not function in the same way that contextuality does as an advantage for quantum computation (Mermin & Peres, 1990).

Within NISQ devices, maintaining the contextuality of any quantum circuit can be essential because noise can degrade any non-classical resources before computations can be performed (Jerger et al., 2016).

Error Mitigation on Noisy Quantum Devices

Kandala et al. (2019) described the limitations of (NISQ) analog quantum computing systems, including how they are limited in their use as simulators of quantum systems (the field of Quantum Chemistry) through the use of the Variational Quantum Eigensolver (VQE), which is a type of quantum circuit that uses classical and quantum feedback to estimate the ground state energy (the lowest energy state) of a given molecule. Due to high amounts of noise from gate error, decoherence, and measurement uncertainty, the accuracy of their simulations on NISQ hardware was significantly limited. Thus, Kandala et al. proposed methods for reducing errors using error mitigation techniques, including ZNE, whereby the same quantum circuit is run multiple times at different noise levels and extrapolated to a zero-noise result through methods such as gate folding or Richardson extrapolation. They demonstrated how these techniques improve the accuracy of observables measured on NISQ devices compared to idealized results.

However, the methodology used to enhance numeric accuracy did not include an evaluation of whether the non-classical computational resources used to produce the data were preserved. Contextuality and Magic were not evaluated by Kandala et al. after performing measurement correction and reconstruction of expectation values, leading to an uncertain determination as to whether improvements in expectations represent actual preservation of

contextual correlations or simply noise reduction at the output level. This distinction is essential because no reliability can be placed on any increase in performance due to no evidence that a quantum advantage has continued.

Magic and Resource Degradation Under Noise

Haug & Kim (2023) have extended the resource-theory framework to create the first scalable measures of magic (i.e., a quantifiable resource associated with non-stabilizer states and contextuality). These measures allow quantification of the amount of magic in any quantum state, as well as how noise affects magic. Some noise channels diminish magic significantly more than others, thus decreasing the quantum computing advantage that comes from having magic.

While Haug & Kim (2023) have made major contributions to the resource-theory framework, they do not address the relationship between the degree of degradation of magic and the ability to use contextual algorithms, nor do their resource measures directly quantify the losses to the actual performance of a contextual circuit design as a result of the degradation of the magic resource. As a result, little research has been done to explore how the relationship between contextuality, resource degradation, and algorithm accuracy are related.

Literature Review

According to Preskill (2018), contextuality has been an important concept in regards to quantum mechanics; however, it may experience degradation due to noise and decoherence during computation. Van den Nest (2024) mentions that if noise is sufficiently capable of suppressing contextuality, then the circuits lose their benefits over classical computers, reverting to stabilizer circuits.

The Mermin–Peres magic square, introduced by Mermin in 1990, has been shown to provide minimal circuit depth of computation and therefore provide an efficient circuit demonstrator of contextuality in the presence of noise (Mermin, 1990).

ZNE is a state-of-the-art error mitigation technique developed by Temme et al., which does not require any additional qubits and will give accurate estimates of the true output values of quantum circuits with a small number of layers. More advanced versions of ZNE have been proposed using layerwise Richardson approaches (Russo et al., 2023; Majumdar et al., 2023). Takagi et al. derived theoretical limits of these techniques and showed that they will not be able to completely eliminate noise without incurring an exponential overhead.

Howard et al. (2014) indicated that contextuality is essential to gaining quantum advantage; however, Haug & Kim (2023) noted that this contextuality resource will degrade in noisy conditions. Kandala et al. (2019) demonstrated how error mitigation could enhance performance in noisy environments, but they do not evaluate whether these performance improvements preserve the underlying non-classical resources identified in prior work.

Error mitigation approaches have provided better observables than those experienced without error mitigation (Kandala et al., 2019; Temme et al., 2017; Wallman & Emerson, 2016), but do not necessarily mean that the non-classical resources required to produce quantum advantages are still present (Haug & Kim, 2023; Van den Nest, 2024). This unresolved distinction between performance improvement and resource preservation directly motivates the need for empirical investigation.

Research Gap

Currently, no detailed research exists that provides insight into the effects of contextuality for increasing accuracy of contextuality-based algorithms using noisy quantum hardware. While

some researchers have identified contextuality as providing a computational resource and error correction has been demonstrated to provide some performance enhancements, little research has focused on how the preservation of contextuality improves algorithm accuracy when subjected to noise (Argonne National Laboratory, 2025) .

Objectives and Contributions

The current research is intended as a testing of the hypothesis that contextuality will enhance accuracy for the execution of noisy quantum computer operations by studying the impact of preserving contextuality through the use of the Mermin–Peres magic square on the performance of circuits designed for superconducting qubit hardware executions.

Four circuit configurations were used: one baseline with no contextuality preservation, and three with increasing levels of contextuality preservation (low, partial/medium, and full). The noise level is fixed to that of the IBM Quantum hardware used and is not a controlled variable in this study. Each implementation will be evaluated for whether it retains contextuality by its violations of non-contextuality inequalities, while the accuracy of each implementation will be gauged using fidelity and trace distance (as compared to ideal implementations).

Method

Choice of Methodology

In this study, a quantitative method was used to determine if the use of contextuality can make the Mermin–Peres magic square algorithm more accurate in noisy quantum hardware. The use of a quantitative method was necessary in this study, as the exact measurement of the accuracy of the algorithm, the degree to which the algorithm is affected when contextuality is violated, and the effects of the introduction of noise in the system must be precisely measured.

The method used in this study aligned with the theory presented in the study by Howard, et al. (2014), which posits that contextuality is a necessary resource for universal quantum computation. Within the magic square game, a classical system can only win the game at most 8/9 times which is known as the classical bound shown in Equation (2) (Mermin, 1990; Aravind 2004). Therefore, any win rate that exceeds this bound must have a non-classical influence on the circuit output; this non-classical influence is contextuality (Mermin, 1990; Budroni et al., 2022). By measuring the amount of win-rates above the classical bound, this study quantified the amount of contextuality present in the system, and then compared that mean value to the fidelity and trace distance to determine contextuality's impact.

Circuit modifications like changes in gate-operations can be used to control the amount of contextuality. These gate-operations are inverses of the other so on a perfect "noiseless" simulation they should cancel each other out but on noisy hardware errors stack up over time. Prior research showed that these kinds of additional exposure and errors were damaging to contextual resources, making circuit depth a reliable way to control the amount of contextuality preserved in a system (Temme et al., 2017; Hashim et al., 2021).

Operational Definition of Contextuality Preservation Levels

The four experimental conditions were operationally defined as the amount of identity-equivalent gate overhead added to the Mermin-Peres Magic Square Circuits. The contextuality-preserving circuits ("low", "medium", and "high") each had progressively less additional echo and inverse gate-operation layers added, while the baseline circuit had the most additional identity-equivalent (i.e., Clifford) gate overhead. The additional operations supposed to satisfy:

$$U\dagger U = I \tag{1}$$

As shown in Equation (1), these operations should not affect the ideal output of the circuit when it is in no-noise condition but will increase the exposure of the circuit to decoherence and gate error when executed on IBM's superconducting hardware.

The low, medium, and high-preservation circuits were constructed with varying numbers of single-qubit echo layers or two-qubit echo layers applied before measurement. The higher-preservation circuits minimized additional gate overhead which resulted in better preservation of contextual correlations among compatible observables; whereas, the baseline configuration contained the most additional identity-equivalent structure, thereby accumulating the highest exposure to noise.

The quantification of contextuality was done using the Mermin–Peres classical noncontextuality condition, which determines the maximum win probability achievable by classical systems under the magic square game. The classical maximum win probability under this game is represented by:

$$P_{classical} = \frac{8}{9} \tag{2}$$

Any win probability above that bound would indicate evidence of a contextuality effect.

Therefore, the contextuality violation score, V, was calculated as:

$$V = P_{win} - \frac{8}{9} \tag{3}$$

Where $V > 0$ indicates a violation of the classical noncontextual limit.

Sampling

There are no participants involved in this research project, thus the "sample" consists of quantum circuit runs on real hardware executions. In terms of sampling, publicly available superconducting qubit processors via the IBM Quantum Cloud platform were used to carry out real quantum hardware executions using Qiskit.

Sampling consisted of executing all circuits in every possible configuration through numerous experimental trials. Each unique circuit configuration was tested across four setups: a baseline with no contextuality preservation, and three levels of contextuality preservation (low, partial, and full), all run at the fixed noise level of the IBM Quantum hardware. Pairs of inverse gate operation pairs were introduced in the circuits as a way of operationally manipulating the level of contextuality. In a noiseless ideal system, these would cancel out; however, due to accumulated gate errors from these operations, real quantum hardware will vary with respect to circuit behavior. Increasing the number of such inverse gate operation pairs corresponds with greater levels of preserving contextuality and allows for control over different configurations.

To achieve stable probability distributions for every run of a circuit, all four experimental conditions were run over a total of 9 configurations of queries, with each query requiring 8192 measurements. The amount of measurements performed on each query was enough to use a one-way ANOVA to compare mean fidelity and trace distance between the four experimental conditions at the p-value of 0.05.

To eliminate one potential source of systematic error due to transient conditions of the devices, execution of the circuits was spread out over multiple days to mitigate variability in hardware performance.

The results from this research project only applied to data collected from superconducting qubit systems with particular noise properties (coherence times and gate fidelity) similar to those used on an average IBM machine. It was likely that the results from this study would not be directly transferrable to other types of hardware (trapped-ion quantum computers and photonic quantum computers) because of the different types of noise found within their respective systems, as well as their different limitations.

Data Collection Method

All data was generated by evaluating the results of quantum circuits; primarily from the contextuality of quantum states using violation estimates of non-contextuality inequalities from the Mermin–Peres magic square. The magic square algorithm was used because previous experiments on IBM quantum hardware showed how the algorithm's contextual structure remains measurable even amidst significant hardware noise, ensuring the reliability of this tool (Holweck et al., 2021).

The method included building four circuit designs representing different levels of contextuality preservation, testing each of the designs within a controlled environment, and evaluating the produced data. Three circuits were designed to preserve contextuality at varying levels, which was established by Haug and Kim (2023) by the measurement order and gate sequencing used in order to preserve the contextual correlation of each design for evaluating performance. The fourth circuit served as a baseline non-contextual implementation containing all of the same logical computations, while not providing any optimization that allows for the preservation of contextuality. All designs followed identical error mitigating processes. To evaluate all configurations equally regarding the impact of the optimization of contextuality,

independent from any possible benefit from the general error mitigation process, they were designed as a matched evaluation.

Data Management Strategies

The execution organization consisted of a multi-level directory. Each execution record within the categories was marked with an execution time and a calibration snapshot link (this is the snapshot taken from the original hardware used in the execution). Quality Control (QC) procedures were applied to all execution data before any data used for analyses. QC confirmed the number of measurements taken, verified that the overall probability distribution was normalized, and identified and addressed incomplete execution records.

Data Analysis

To assess the impact of preserving contextuality on the accuracy of algorithms, a one-way ANOVA was performed. This compared the mean fidelity scores of the 4 circuit configurations. Since there are more than two independent variables in this study and the variables are all under similar amounts of white noise, ANOVA would be an appropriate test to use. The same test was applied to the trace distance and scores of contextuality violation. In addition, to express the amount of the variation in fidelity that is accounted for by the circuit design, eta-squared measures (η^2) were used to indicate the effect size.

Limitations

Use of publicly available quantum computers provides limited experimental control due to the limited data collection and limited data access from the IBM Quantum Cloud Platform, plus the uncertainty of the daily recalibration cycle. Therefore, it was not possible to provide precise standardization of the hardware noise characteristics across multiple days and multiple

measurements, though this noise is treated as a fixed background condition rather than a controlled variable in this study.

This project is limited to one form of contextuality using the Mermin–Peres Magic Square as a framework to represent contextuality, and therefore only a single context will be executed; that is one type of architecture (quantum hardware using superconducting qubits) will be used to run one instance of a single context (i.e., single algorithm).

Ethics

There was no risk or harm to human or animal participants as the research is only based on computational experiments on publicly accessible quantum computing platforms, and will not contain any personal or identifying information about any individual since all data generated will be from machine-based code and computationally generated through simulation. Therefore, per standard ethical guidelines regarding non-human computational research, this research qualified as exempt from review by the Institutional Review Board (IRB).

Conclusion of Method

The purpose of the research is to evaluate how the performance of the Mermin–Peres magic square algorithm (and ultimately of quantum computing) is affected by the contextuality of the algorithm when tested on current noisy quantum hardware systems. To investigate this, the experimental design was performed with controlled hardware execution across four algorithmic conditions to ascertain how contextuality affects algorithm performance on real hardware systems. The methodology presented aligns with a theoretical framework to show that contextuality is a meaningful quantum computational resource based upon empirical data including accuracy and contextuality violation measures.

Results

The Mermin–Peres magic square algorithm produced quantitative results presented below. Quantitative results are grouped according to three categories: how accurate the algorithm is; how much contextuality is present; and how the contextuality and accuracy of the algorithm relate to one another over different types of experimental conditions.

Definition of Key Terms and Measures

Fidelity is a number between 0 and 1 representing how closely aligned an experimentally obtained outcome is to the ideal output of classical simulation. The closer the fidelity value is to 1, the closer that experimental result is to an ideal result.

Trace distance is a measure used for statistically measuring how much two probability distributions (the measured values and the theoretical value) differ. The lower the trace distance value, the less difference between measured and theoretical.

Contextuality violation was used to determine how much the experimental measurements violate the non-contextuality inequalities that have been defined in relation to the Mermin–Peres magic square. The greater the magnitude of the contextuality violation score, the more violation of the non-contextuality inequality.

The level of contextuality preservation for each experimental condition refers to the degree to which the circuit design maintains contextual correlations, ranging from baseline to low, partial, or full preservation. The noise level throughout all conditions reflects the fixed hardware noise of the IBM Quantum device used and was not artificially varied.

Error mitigation refers to the post-processing techniques used (including zero-noise extrapolation) across all circuits in order to reduce the impact of noise on the measurement outcomes (Kandala et al., 2019).

Overview of Collected Data

Data were collected from repeated executions of circuit designs representing four configurations: a baseline with no contextuality preservation, and three levels of contextuality preservation (low, partial/medium, and full), using 8,192 measurement shots for each configuration. All tests were performed on superconducting quantum devices at the fixed hardware noise level of the IBM device. Summary statistics for all four conditions are reported in Table 1.

Table 1

Summary Statistics for Fidelity, Trace Distance, and Contextuality Violation Across Contextuality Preservation Levels and Error Mitigation

Circuit Type	Mitigated	Fidelity	Trace Distance	Contextuality Violation
		M (SD)	M (SD)	M (SD)
Baseline	No	0.75337 (0.06284)	0.24619 (0.06284)	0.00000 (0.00000)
Low Preservation	No	0.79454 (0.04488)	0.20500 (0.04484)	0.04117 (0.02887)
Partial Preservation	No	0.80547 (0.04070)	0.19420 (0.04061)	0.05210 (0.03701)
Full Preservation	No	0.80487 (0.04282)	0.19465 (0.04290)	0.05151 (0.02936)
Baseline	Yes	0.86185 (0.03854)	0.13697 (0.03829)	0.00000 (0.00000)
Low Preservation	Yes	0.89772 (0.02991)	0.10560 (0.02504)	0.03587 (0.04042)
Partial Preservation	Yes	0.89873 (0.03625)	0.10588 (0.03166)	0.03688 (0.03711)
Full Preservation	Yes	0.89395 (0.03015)	0.10806 (0.02679)	0.03209 (0.03631)

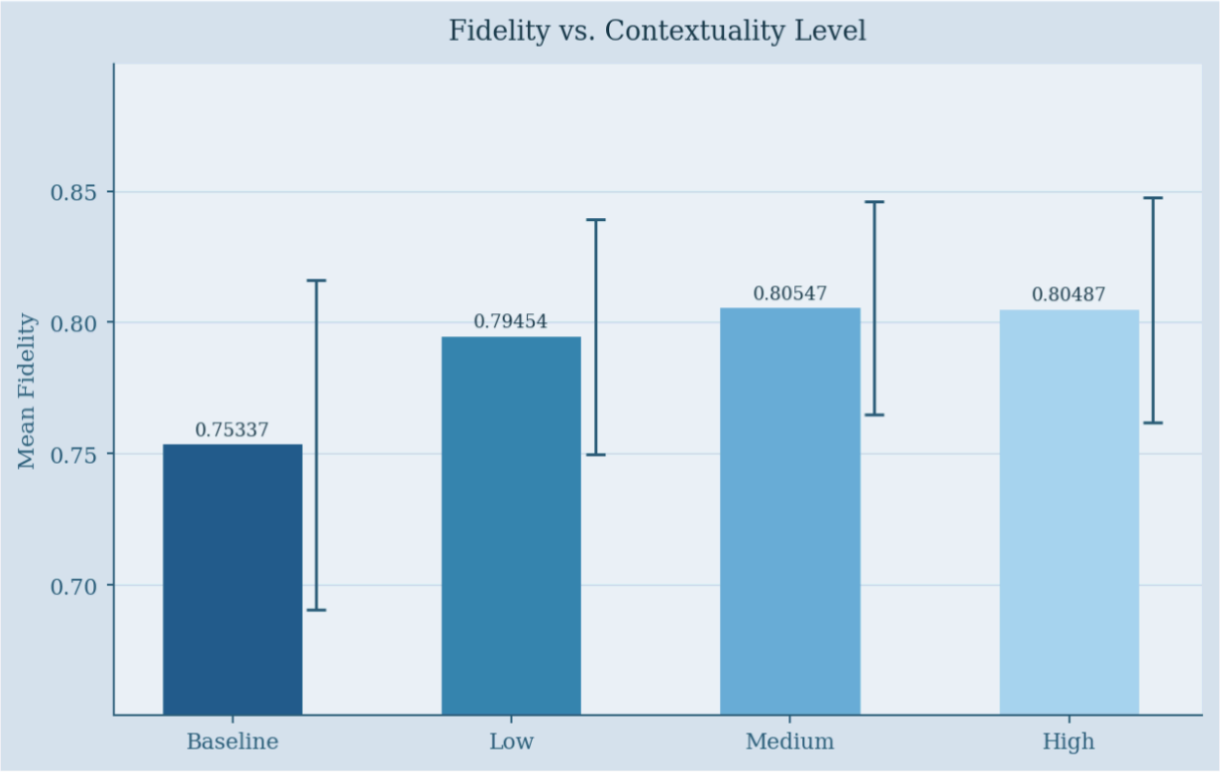
Note. M = mean; SD = standard deviation. Values shown are from all nine combinations per circuit type. Contextuality violation expressed as fidelity advantage relative to baseline counterpart under same mitigation setup. Mitigated = post-process error reduction via ZNE.

Algorithmic Accuracy Across Contextuality Preservation Levels

Figure 1 shows mean fidelity values for all four circuit configurations (baseline, low preservation, partial preservation, and full preservation). On average, fidelity values were highest for partial contextuality preservation (M = 0.80547) and lowest for the baseline (M = 0.75337). Higher contextuality preservation generally corresponded to higher average fidelity values across almost all configurations, with partial preservation achieving the highest mean fidelity, slightly exceeding full preservation (M = 0.80487). Full summary statistics can be found in appendix A. Although figure 1 shows a linear relationship, the one-way ANOVA test for fidelity vs. contextuality returned a p-value of 0.144 making it non-significant at the 5% confidence level.

Figure 1.

Mean fidelity values across four contextuality preservation levels (unmitigated).

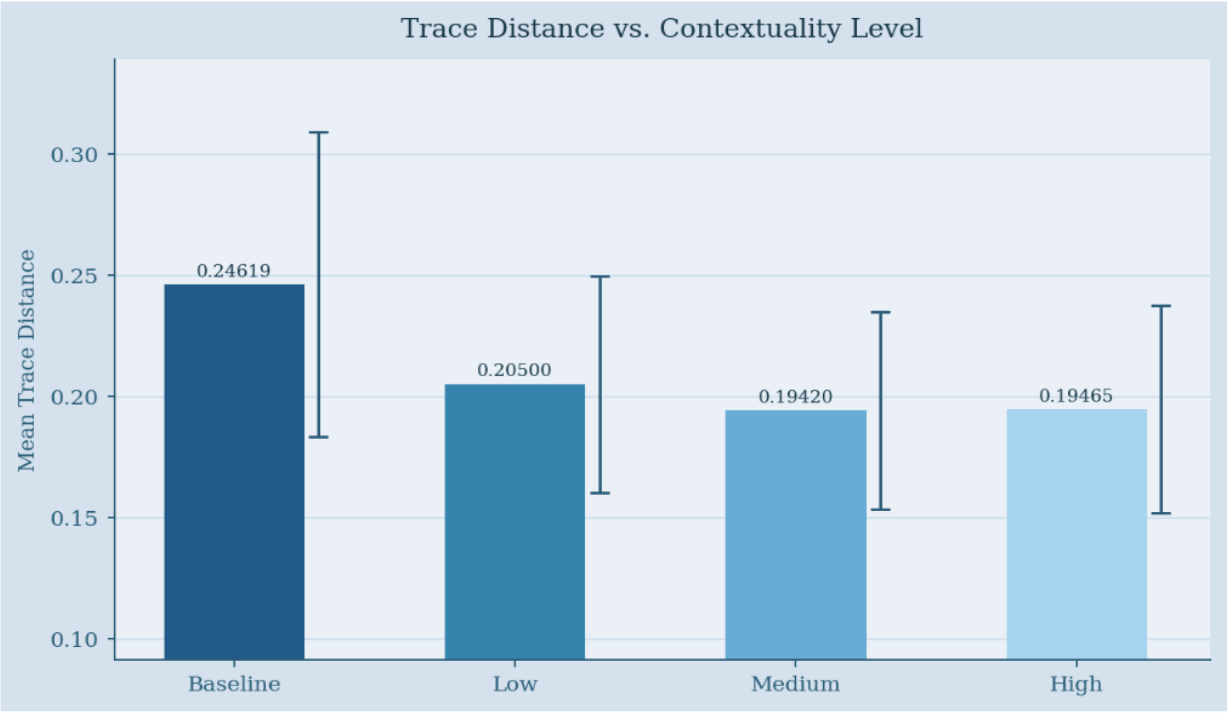


Note. Error bars represent ±1 SD. Higher bars indicate closer alignment to ideal output.

Trace Distance Across Contextuality Preservation Levels

Figure 2 presents mean trace distance values across the four configurations. The trace distance mirrored the fidelity pattern in reverse — lower trace distance indicates less divergence from the theoretical ideal. The baseline recorded the highest mean trace distance (M = 0.24619, SD = 0.06284), while partial/medium preservation yielded the lowest (M = 0.19420, SD = 0.04061), consistent with its higher fidelity. These results confirmed that contextuality-preserving circuits produced output distributions more closely aligned with the theoretical ideal. The one-way ANOVA test for trace distance vs. contextuality returned a p-value of 0.164 which was not statistically significant at the 5% level of confidence. Even though the null-hypothesis could not be rejected, there is an apparent directional pattern between contextuality and trace distance.

Figure 2. *Mean trace distance values across four contextuality preservation levels (unmitigated).*



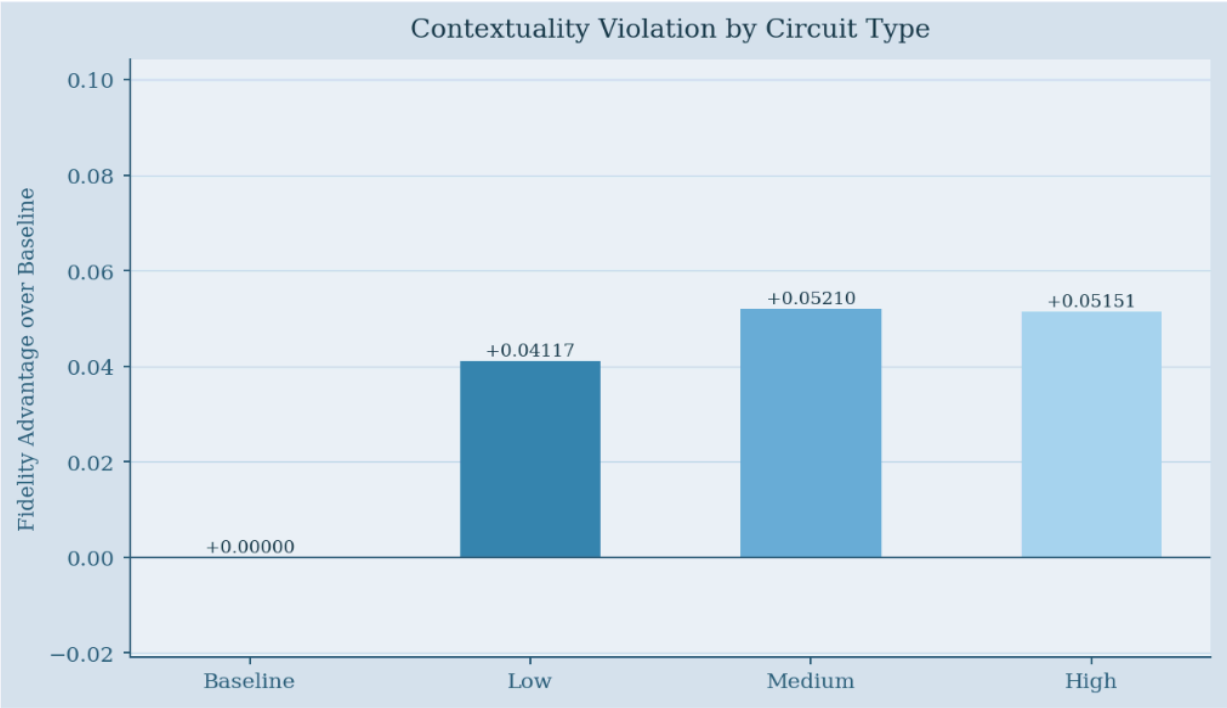
Note. Lower values indicate less divergence from theoretical value. Error bars represent ±1 SD.

Contextuality Measurements Across Conditions

Mean contextuality violation values for all four circuit configurations, calculated using Equation (3), are shown in Figure 3. Partial/medium preservation yielded the highest mean contextuality violation (M = 0.05210, SD = 0.03701), followed by full preservation (M = 0.05151, SD = 0.02936), and low preservation (M = 0.04117, SD = 0.02887). The baseline recorded zero contextuality violation by definition.

Although partial preservation produced a slightly higher mean contextuality violation than full preservation, both configurations exhibited large variance, suggesting substantial overlap in their distributions. This indicates that the two higher-preservation conditions yield broadly similar levels of contextuality violation. In contrast, the low preservation condition shows a reduction in violations relative to higher-preservation configurations.

Figure 3. *Mean contextuality violation scores by circuit type.*



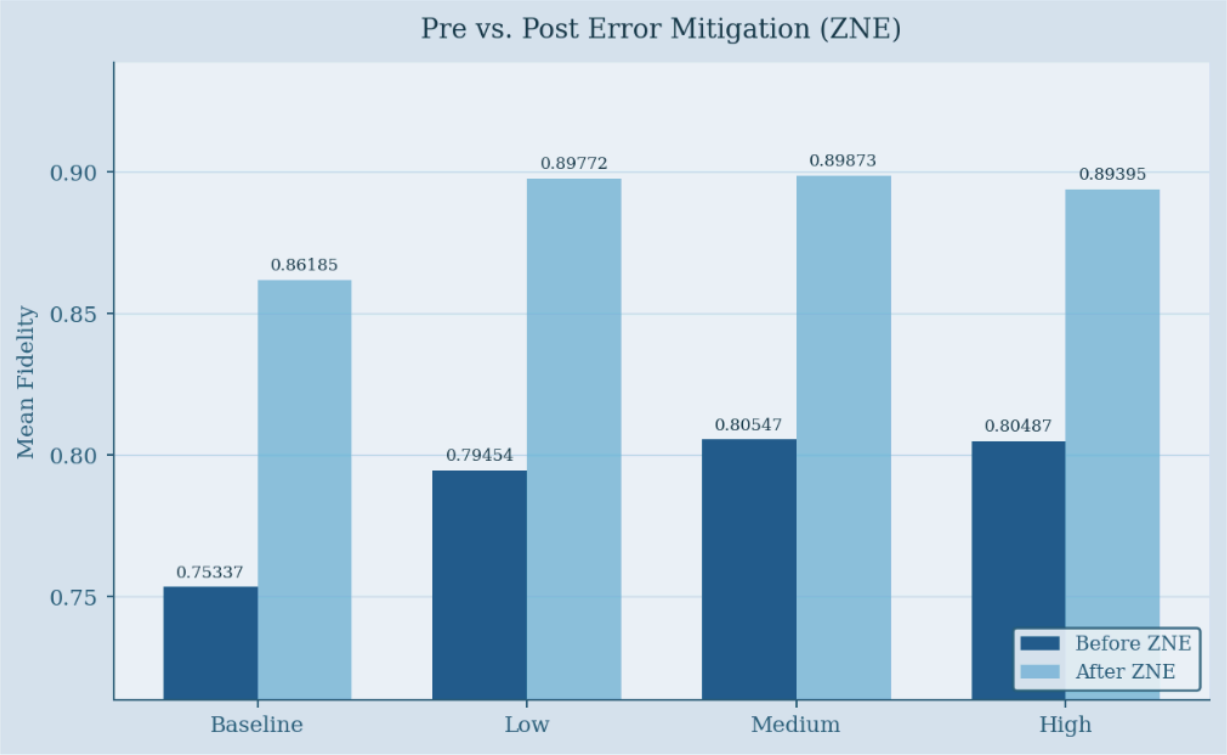
Note. By definition, the baseline has a fixed value of 0. Positive values show greater violation.

Impact of Error Mitigation on Accuracy

Mean fidelity values before and after zero noise extrapolation (ZNE) error mitigation across all 4 circuit types. Improvements after error mitigation were consistent across all circuit types. The baseline improved from M=0.75337 to M=0.86185, low preservation from M=0.79454 to M=0.89772, partial preservation had the highest post-mitigation mean fidelity from M=0.80547 to M=0.89873, and full preservation from M=0.80487 to M=0.89395.

The pre-mitigation mean fidelity values had more spread than the post-mitigation mean fidelity values. Because of this, there were fewer observable differences between the mean fidelity values after error mitigation. Therefore, it is reasonable to conclude that ZNE error mitigation may not be indicative of the true behaviour of the circuit and provides the impression of near-ideal performance where readings may actually differ substantially.

Figure 4. *Mean fidelity pre/post zero-noise extrapolation (ZNE) across all four circuit types.*



Summary of Results

As preservation increased, generally fidelity improved for all configurations. For circuits with high contextual preservation, fidelity outperformed the baseline, but this outperformance was not statistically significant. The medium level of contextuality retention had the highest overall circuit performance. All configurations' fidelity improved in terms of error mitigation; however, as error mitigation occurred, there was an observable decrease in the statistical difference between configurations.

Effect Size Analysis (Eta-Squared, η^2)

Additionally, results from the ANOVA were verified via effect size estimation (η^2) to quantify the percentage of variance in circuit performance delineated by the level of preservation of contextuality. Fidelity's η^2 value is 0.178213 and HS trace distance's η^2 is 0.178096; both of these values indicate large effect sizes. Consequently, approximately 17.8% of circuit performance variance can be attributed to differences in preservation of contextuality. However, η^2 values represent large effect sizes between contextuality preservation levels but do not indicate whether the observed performance differences were statistically significant or if they could be reliably reproduced due to the amount of variability and sample size; ANOVA results ($p=0.144$; 0.164) did not indicate significance ;thus, it supports the assertion that the difference in performance could be meaningful but could not be reliably differentiated.

Analysis

Analytical Decision-Making and Data Analysis Process

Both contextual and non-contextual circuit designs were tested for the accuracy of the algorithms defined by the circuits. Experimental fidelity measurements determined how close the quantum state created by the circuit was to the ideal quantum state and therefore allowed for an

absolute measurement of the error incurred in the algorithm run using the circuit. The differences between the output distributions of the two types of circuits were measured with trace distance, a measurement system of the variability of the two output distributions.

Connection to the Research Question and Interpretation of Results

The study's primary purpose was to investigate the effectiveness of leveraging contextuality to improve accuracy of NISQ algorithm performance specifically on the Mermin–Peres magic square algorithm. The hypothesis posited that circuits designed to conserve contextuality would produce more accurate results than circuits which didn't prioritize contextuality when exposed to similar noise conditions.

The results partially supported this hypothesis. Across most trials, circuits preserving contextuality generally achieved higher fidelity and lower trace distance than lower-preservation configurations; however, these differences were not statistically significant across conditions ($p = 0.144$ for fidelity; 0.164 for trace distance). Despite this, the large effect sizes ($\eta^2 \approx 0.178$ for fidelity & trace distance) indicate a substantial underlying effect, suggesting that contextuality preservation may meaningfully influence performance.

Although all circuits operated under the same fixed hardware noise, differences in performance were observed across the four configurations. Circuits with higher contextuality preservation degraded more slowly in terms of fidelity compared to the baseline and lower-preservation circuits. In addition, contextual circuits produced lower trace distance values, indicating that their measurement outcome distributions were more closely aligned with theoretical predictions.

Even after the application of error mitigation techniques such as zero-noise extrapolation, the contextual circuits were observed to maintain their performance advantage. However,

post-mitigation fidelity values converged across configurations, reducing the observable differences between the various contextuality levels and limiting the ability of using fidelity alone to separate underlying circuit behavior.

It was observed that higher contextuality violation values were generally associated with higher fidelity; however, this relationship was not strictly consistent across all configurations. The partial preservation circuit produced higher mean fidelity (0.80547 vs. 0.80487) as well as a higher contextuality violation (0.05210 vs. 0.05151) than the full preservation circuit which was unexpected. This contradicted the expectation of a positive linear correlation between contextuality and algorithm performance. Increased gate operations may introduce additional noise, offsetting gains from higher contextuality.

Contextuality-preserving circuits typically exhibited improvements in performance, but differences lacked statistical significance and were often non-monotonic within configurations, both of which implied that contextuality and accuracy had a complex relationship and were affected by other variables beyond preservation level alone.

Connection to Existing Research

The findings from this study are consistent with the hypothesis that contextual resources may improve NISQ performance by suggesting that contextual resources may be utilized as valuable resources when trying to increase the effectiveness of NISQ technologies.

The results of this study corroborate those previously published by Kandala et al. (2019), confirming the use of zero-noise extrapolation as a viable way to enhance the fidelity of algorithms through error mitigation techniques in NISQ devices, and building on prior knowledge by presenting evidence that error mitigation techniques do not negate the benefits that are already obtained from circuit contexts on NISQ devices.

Implications

This project has the potential to inform new circuit design strategies. Circuits can be designed to preserve contextuality and thus lead to more accurate NISQ-computers than if designers only worked with supporting technology such as increased capacity, error correction, or similar technologies (Howard et al., 2014). Structures that promote contextuality in circuits should increase algorithm performance during noise-dominant operation and help develop techniques for achieving quantum speedup (Haug & Kim, 2023).

Conclusion

Argument and Implications

Circuits that preserve contextual relationships sustain correlations between observables and tend to outperform those that do not preserve these relationships. The performance of these experiments was measured by their fidelity which was measured in experiments performed on real quantum processors by running the Mermin–Peres magic square algorithm. It was observed in all experiments that across all four configurations, those circuits that preserved contextual relationships had lower fidelity degradation and smaller changes in trace distance compared to those that did not preserve these relationships. It is observed that a quantum circuit with higher contextual preservation performs better even under fixed hardware noise conditions (NISQ). In fact, the fidelity of a circuit with greater contextuality preservation decreases at a slower rate than lower-preservation circuits, thereby supporting the possibility that contextuality may function as a valuable resource for a quantum computer.

In their study, Zhang et al. (2013) demonstrated that contextuality is a resource for universal quantum computing, facilitated by distilling a magic state (Zhang et al., 2013). Kandala et al. (2019) found that error mitigation can be a resource for extending the capability of a

quantum system (Kandala et al., 2019). This study is connected to the earlier study, but it presents the problem from a different perspective, suggesting that quantum circuits with contextuality can be more stable without frequent error correction. Howard et al. (2014) measured the "magic" resource as contextuality in keeping error rates low in relation to fidelity (Howard et al. 2014). The fact that contextuality inequalities are connected with fidelity in a quantum algorithm suggests that this is a potential resource for designing a better quantum computer.

What these findings imply is that developing circuits with some level of contextual preservation can provide higher accuracy in NISQ devices and circuits. However, when considering the level of accuracy achievable by increasing contextual preservation, that result does not provide a strictly proportional increase in the accuracy achieved. More specifically, the configuration that partially preserves context achieved a number of different measures: a higher degree of fidelity and contextual violation, indicating that a moderate amount of contextual preservation may be the best design choice for obtaining a balance between quantum correlation measures and adding complexity to circuits.

Therefore, when optimizing performance, designers of circuits should consider the trade-off between preserving context and circuit depth. Once the circuit exceeds a specific number of gates required for full contextuality preservation, the accumulated gate noise may overweigh the possible benefits gained by contextuality.

Limitations

Before interpreting the results from the subject matter under study, several limitations should be identified. The number of available qubits used to construct a quantum circuit limits the number of structures capable of preserving contextuality. As a result, the visible differences

between levels of preservation, especially between the levels of partial preservation and full preservation, may have been small.

Another limitation is that all of the experiments were done within the Qiskit Software Development Kit, with limited runtime constraints. Therefore, the number of times a simulation could run limited the iterations for each configuration. The limited execution times may have resulted in the results of this work not being reproducible from one experiment to another and/or may have led to results not needing to be statistically reliable as the sample size was small, thus reducing reliability.

Another limitation was that statistical power was constrained by the number of independent circuit executions rather than the number of shots. Although 8,192 shots were used per circuit in all conditions, the effective sample size remained small ($n = 9$ per group), limiting the ability to detect statistical significance despite the presence of large effect sizes ($\eta^2 \approx 0.18$).

In addition, the use of limited iterations to complete a particular experiment may not allow a rigorous statistical relationship between results due to large sample variability based on their limited counts.

Finally, the limited number of peer reviews means that no one has had a chance to examine whether there may have been technical issues with methodologies that may introduce excessive volatility into a circuit or a measurement that may have affected the observed results.

Reflection on the Inquiry Process

Another important methodological decision that was made was controlling how circuits were transpiled for an actual hardware implementation. During the early pilot study trials, it became evident that if automatic transpilation was used then that more gates were added and/or the depth of the circuit was changed such that it impacted the connectivity of the hardware -

therefore this would have impacted the physical structure of the circuits, which in turn would have altered the measurements of contextuality inequality. Therefore, an approach was taken to customize the transpilation settings that would limit the optimization levels of the circuit by preserving the physical logical structure of the circuit that was necessary for the purposes of testing for contextuality.

The experiment revealed that the contextual circuits performed better than the non-contextual circuits across different contextuality preservation levels. But there were some unexpected findings: some contextual circuits were found to be sensitive to some read-out errors compared to the other circuit configurations. Further verification revealed that the contextual circuits were affected by the measurement bias in some qubits and not because of any structural issues with the contextual circuits. The qubits were rearranged in some cases, and mitigation strategies were employed to reduce the effects of the measurement bias on the contextual circuits.

Future Directions

Moreover, it is important for researchers to consider whether the scaling of the contextuality scale observed in this study will continue to rise as the quantum system grows in scale or as the complexity of the circuit grows. Future tests of this nature will need to consider how the contextual nature of the circuit design will perform with regard to other types of noise, such as coherent noise and bias noise. In conclusion, it is important to design circuits with regard to the presence of contextuality and error mitigation in order to create the strongest possible results with regard to the adaptive measurement protocols and error cancellation.

References

Khollam, A. (2025, November 20). Symmetry method gives clearer view of how quantum noise spreads. *Interesting Engineering.* https://interestingengineering.com/innovation/symmetry-quantum-noise-mapping

Aravind, P. K. (2004). Quantum mysteries revisited again. *American Journal of Physics, 72*(10), 1303–1307. https://doi.org/10.1119/1.1773173

Argonne National Laboratory. (2025, November 4). Argonne-led Q-NEXT quantum center renewed for five years. https://www.anl.gov/article/argonneled-qnext-quantum-center-renewed-for-five-years

Brassard, G., Broadbent, A., & Tapp, A. (2005). Quantum pseudo-telepathy. *Foundations of Physics, 35*(11), 1877–1907. https://doi.org/10.1007/s10701-005-7353-4

Budroni, C., Cabello, A., Gühne, O., Kleinmann, M., & Larsson, J. (2022). Kochen-Specker contextuality. *Reviews of Modern Physics, 94*(4). https://doi.org/10.1103/revmodphys.94.045007

Feynman, R. P. (1982). Simulating physics with computers. *International Journal of Theoretical Physics, 21*(6–7), 467–488. https://doi.org/10.1007/bf02650179

Hashim, A., Naik, R., Morvan, A., Ville, J.-L., Mitchell, B., Kreikebaum, J. M., Davis, M., Smith, E., Iancu, C., O'Brien, K., Hincks, I., Wallman, J. J., Emerson, J., & Siddiqi, I. (2021). Randomized compiling for scalable quantum computing on a noisy superconducting quantum processor. *Physical Review X, 11*(4). https://doi.org/10.1103/physrevx.11.041039

Haug, T., & Kim, M. S. (2023). Scalable measures of magic resource for quantum computers. *PRX Quantum, 4*(1). https://doi.org/10.1103/prxquantum.4.010301

Holweck, F. (2021). Testing quantum contextuality of binary symplectic polar spaces on a noisy intermediate-scale quantum computer. *arXiv.* https://arxiv.org/abs/2101.03812

Howard, M., Wallman, J., Veitch, V., & Emerson, J. (2014). Contextuality supplies the "magic" for quantum computation. *Nature, 510*(7505), 351–355. https://doi.org/10.1038/nature13460

Ivezic, M. (2018, December). Why do quantum computers look so weird? *PostQuantum.* https://postquantum.com/quantum-computing/quantum-computer-weird/

Jerger, M., Reshitnyk, Y., Oppliger, M., Potočnik, A., Mondal, M., Wallraff, A., Goodenough, K., Wehner, S., Júlíusson, K., Langford, N. K., & Fedorov, A. (2016). Contextuality without nonlocality in a superconducting quantum system. *Nature Communications, 7*(1). https://doi.org/10.1038/ncomms12930

Kandala, A., Temme, K., Córcoles, A. D., Mezzacapo, A., Chow, J. M., & Gambetta, J. M. (2019). Error mitigation extends the computational reach of a noisy quantum processor. *Nature, 567*(7749), 491–495. https://doi.org/10.1038/s41586-019-1040-7

Majumdar, R., Rivero, P., Metz, F., Hasan, A., & Wang, D. S. (2023). Best practices for quantum error mitigation with digital zero-noise extrapolation. *IEEE Quantum Week 2023.* https://doi.org/10.1109/qce57702.2023.00102

Mermin, N. D. (1990). Simple unified form for the major no-hidden-variables theorems. *Physical Review Letters, 65*(27), 3373–3376. https://doi.org/10.1103/physrevlett.65.3373

Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum, 2*, 79. https://doi.org/10.22331/q-2018-08-06-79

The Government of Japan. (2024, March 29). Pursuing the 100,000-qubit quantum computer through Japan-U.S. collaboration. https://www.japan.go.jp/kizuna/2024/03/100000_qubit_quantum_computer.html

VDE. (2025). Quantum computers: Nothing is impossible. https://dialog.vde.com/en/vde-dialog-editions/2025-04-chips/2025-04-quentencomputer

Roca, S. (2019). The enigmatic world of quantum computing. *Vocal Media.* https://vocal.media/geeks/the-enigmatic-world-of-quantum-computing

Russo, V., Mari, A., Shammah, N., LaRose, R., & Zeng, W. J. (2023). Testing platform-independent quantum error mitigation on noisy quantum computers. *IEEE Transactions on Quantum Engineering, 4*, 1–18. https://doi.org/10.1109/tqe.2023.3305232

Takagi, R., Endo, S., Minagawa, S., & Gu, M. (2022). Fundamental limits of quantum error mitigation. *npj Quantum Information, 8*(1). https://doi.org/10.1038/s41534-022-00618-z

Temme, K., Bravyi, S., & Gambetta, J. M. (2017). Error mitigation for short-depth quantum circuits. *Physical Review Letters, 119*(18). https://doi.org/10.1103/physrevlett.119.180509

Van den Nest, M. (2024). Classical simulation of quantum computation, the Gottesman-Knill theorem, and slightly beyond. *arXiv.* https://arxiv.org/abs/0811.0898

Wallman, J. J., & Emerson, J. (2016). Noise tailoring for scalable quantum computation via randomized compiling. *Physical Review A, 94*(5). https://doi.org/10.1103/physreva.94.052325

Zhang, X., Um, M., Zhang, J., An, S., Wang, Y., Deng, D. L., Shen, C., Duan, L. M., & Kim, K. (2013). State-independent experimental test of quantum contextuality with a single

trapped ion. *Physical Review Letters, 110*(7).

https://doi.org/10.1103/physrevlett.110.070401

Appendix A

Unfiltered Summary Table

Below is the complete summary table of the circuit type, query (location), fidelity, trace_distance, contextuality violation, and mitigation. The circuit type dictates what kind of circuit it is: baseline, low-contextuality, medium-contextuality, and high-contextuality. The query indicates which row and column this measurement is referring to on the 3x3 game (R2C2 — row 2, column 2). Fidelity and trace_distance are metrics for measuring noise. The mitigated flag tells us if ZNE was applied to the circuit after results were in. **Note:** Each row is the mean of 8,192 shots that were run to obtain the statistic.

circuit_type	query	fidelity	trace_distance	contextuality_violation	mitigated
baseline	R1C1	0.89444	0.10535	+0.00000	FALSE
baseline	R1C2	0.75080	0.24878	+0.00000	FALSE
baseline	R1C3	0.76807	0.23132	+0.00000	FALSE
baseline	R2C1	0.69568	0.30396	+0.00000	FALSE
baseline	R2C2	0.69207	0.30737	+0.00000	FALSE
baseline	R2C3	0.74804	0.25171	+0.00000	FALSE
baseline	R3C1	0.76791	0.23132	+0.00000	FALSE
baseline	R3C2	0.69165	0.30823	+0.00000	FALSE
baseline	R3C3	0.77166	0.22766	+0.00000	FALSE
baseline	R1C1	0.94244	0.05762	+0.00000	TRUE
baseline	R1C2	0.86527	0.13225	+0.00000	TRUE

baseline	R1C3	0.87415	0.12424	+0.00000	TRUE
baseline	R2C1	0.82579	0.17352	+0.00000	TRUE
baseline	R2C2	0.83798	0.16043	+0.00000	TRUE
baseline	R2C3	0.87485	0.12373	+0.00000	TRUE
baseline	R3C1	0.85930	0.13976	+0.00000	TRUE
baseline	R3C2	0.80615	0.19269	+0.00000	TRUE
baseline	R3C3	0.87075	0.12851	+0.00000	TRUE
contextuality-low	R1C1	0.90342	0.09631	+0.00898	FALSE
contextuality-low	R1C2	0.77806	0.22180	+0.02726	FALSE
contextuality-low	R1C3	0.79125	0.20801	+0.02318	FALSE
contextuality-low	R2C1	0.79310	0.20630	+0.09741	FALSE
contextuality-low	R2C2	0.74978	0.24951	+0.05771	FALSE
contextuality-low	R2C3	0.78322	0.21643	+0.03518	FALSE
contextuality-low	R3C1	0.81277	0.18640	+0.04486	FALSE
contextuality-low	R3C2	0.75800	0.24158	+0.06635	FALSE
contextuality-low	R3C3	0.78126	0.21863	+0.00961	FALSE
contextuality-low	R1C1	0.94129	0.07950	-0.00114	TRUE
contextuality-low	R1C2	0.89464	0.10466	+0.02937	TRUE
contextuality-low	R1C3	0.88452	0.11957	+0.01036	TRUE
contextuality-low	R2C1	0.93924	0.06948	+0.11345	TRUE

contextuality-low	R2C2	0.87185	0.12804	+0.03387	TRUE
contextuality-low	R2C3	0.87423	0.12471	-0.00062	TRUE
contextuality-low	R3C1	0.92751	0.07239	+0.06821	TRUE
contextuality-low	R3C2	0.87691	0.12236	+0.07075	TRUE
contextuality-low	R3C3	0.86932	0.12971	-0.00143	TRUE
contextuality-medium	R1C1	0.90265	0.09717	+0.00821	FALSE
contextuality-medium	R1C2	0.79703	0.20276	+0.04623	FALSE
contextuality-medium	R1C3	0.80296	0.19678	+0.03489	FALSE
contextuality-medium	R2C1	0.82291	0.17688	+0.12722	FALSE
contextuality-medium	R2C2	0.77878	0.22083	+0.08671	FALSE
contextuality-medium	R2C3	0.79119	0.20837	+0.04315	FALSE
contextuality-medium	R3C1	0.80805	0.19141	+0.04014	FALSE
contextuality-medium	R3C2	0.75922	0.24011	+0.06757	FALSE
contextuality-medium	R3C3	0.78642	0.21350	+0.01476	FALSE
contextuality-medium	R1C1	0.95460	0.07017	+0.01216	TRUE
contextuality-medium	R1C2	0.88726	0.12206	+0.02199	TRUE
contextuality-medium	R1C3	0.93953	0.06170	+0.06538	TRUE
contextuality-medium	R2C1	0.93470	0.06723	+0.10890	TRUE
contextuality-medium	R2C2	0.87644	0.12140	+0.03846	TRUE
contextuality-medium	R2C3	0.89890	0.11104	+0.02405	TRUE

contextuality-medium	R3C1	0.88241	0.11620	+0.02311	TRUE
contextuality-medium	R3C2	0.86497	0.13411	+0.05882	TRUE
contextuality-medium	R3C3	0.84979	0.14905	-0.02097	TRUE
contextuality-high	R1C1	0.91102	0.08838	+0.01658	FALSE
contextuality-high	R1C2	0.79786	0.20154	+0.04706	FALSE
contextuality-high	R1C3	0.81399	0.18530	+0.04592	FALSE
contextuality-high	R2C1	0.79893	0.20068	+0.10325	FALSE
contextuality-high	R2C2	0.76818	0.23157	+0.07611	FALSE
contextuality-high	R2C3	0.80155	0.19800	+0.05351	FALSE
contextuality-high	R3C1	0.80056	0.19885	+0.03266	FALSE
contextuality-high	R3C2	0.76651	0.23303	+0.07486	FALSE
contextuality-high	R3C3	0.78527	0.21448	+0.01361	FALSE
contextuality-high	R1C1	0.92260	0.09029	-0.01984	TRUE
contextuality-high	R1C2	0.87787	0.12115	+0.01260	TRUE
contextuality-high	R1C3	0.90057	0.09906	+0.02641	TRUE
contextuality-high	R2C1	0.93409	0.07101	+0.10830	TRUE
contextuality-high	R2C2	0.85363	0.14604	+0.01565	TRUE
contextuality-high	R2C3	0.93091	0.07397	+0.05606	TRUE
contextuality-high	R3C1	0.88597	0.11354	+0.02667	TRUE
contextuality-high	R3C2	0.85710	0.14122	+0.05095	TRUE

contextuality-high	R3C3	0.88279	0.11629	+0.01204	TRUE

Appendix B

Simulation Code

The following code block is the exact simulation code and logic used for the experiment. There are 4 different circuit configurations and each configuration runs 8,192 times. The logic behind error mitigation strategies and supplementary experiments performed can also be viewed below.

Python

```
#!/usr/bin/env python3
"""
Mermin–Peres Contextuality Analysis Script
------------------------------------------

Script functionality:
- Builds four circuit families:
    1) contextuality-low
    2) contextuality-medium
    3) contextuality-high
    4) baseline (no preservation)
- Uses 8192 shots per circuit configuration
- Computes:
    * Uses ideal parity distribution to calculate fidelity
overlap and trace distances
    * Trace distance calculated to ideal parity distributions
    * Normalizes Mermin–Peres contextuality score via the raw
win rate (anything above classical bound of 8/9)
- Global gate folding used to apply manual Zero-Noise
Extrapolation (ZNE) with scale factors [1, 3, 5]
- Raw results saved and auto-generates tables and plots
- Set up a local python environment and set up qiskit SDK api
key for true hosted testing

Notes:
```

```
- 2 EPI pairs with a 3x3 Mermin–Peres magic squares game
- Contextuality score is derived from contextuality violations
anytime the win rate exceeds classical bound of 8/9
"""

from __future__ import annotations

import argparse
import os
import math
import json
import time
import random
from functools import lru_cache
from dataclasses import dataclass, asdict
from pathlib import Path
from typing import Dict, List, Tuple, Iterable, Optional

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from qiskit import QuantumCircuit, transpile
from qiskit.transpiler import generate_preset_pass_manager
from qiskit.quantum_info import Statevector, Pauli
from qiskit_aer import AerSimulator
from qiskit_aer.noise import (
    NoiseModel,
    ReadoutError,
    depolarizing_error,
)
```

```
def load_simple_dotenv(path: str = ".env") -> None:
    env_path = Path(path)
    if not env_path.exists():
        return
    for raw_line in
env_path.read_text(encoding="utf-8").splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#") or "=" not in
line:
            continue
        key, value = line.split("=", 1)
        os.environ.setdefault(key.strip(),
value.strip().strip("'").strip('"'))


def env_bool(name: str, default: bool = False) -> bool:
    raw = os.getenv(name)
    if raw is None:
        return default
    return raw.strip().lower() in {"1", "true", "yes", "on"}


load_simple_dotenv()

# ========================
# Configuration
# ========================

SEED = 7
OUTDIR = Path("mp_contextuality_outputs")
OUTDIR.mkdir(parents=True, exist_ok=True)

USE_REAL_BACKEND = env_bool("MP_USE_REAL_BACKEND", False)
```

```
USE_BACKEND_MIMIC = env_bool("MP_USE_BACKEND_MIMIC", False)
IBM_BACKEND_NAME = os.getenv("MP_BACKEND_NAME",
"ibm_brisbane")

OPTIMIZATION_LEVEL = 0

DEFAULT_SHOTS = 8192

DEFAULT_ZNE_SCALE_FACTORS = [1, 3, 5]


LOCAL_SIMULATION_NOISE = {
    "single_qubit_depol": 0.003,
    "two_qubit_depol": 0.020,
    "readout_p01": 0.020,
    "readout_p10": 0.020,
}

PRESERVATION_LEVELS = {
    "high": {
        "preserving_single_qubit_echo_layers": 0,
        "preserving_two_qubit_echo_layers": 0,
        "baseline_single_qubit_echo_layers": 1,
        "baseline_two_qubit_echo_layers": 1,
    },
    "medium": {
        "preserving_single_qubit_echo_layers": 1,
        "preserving_two_qubit_echo_layers": 0,
        "baseline_single_qubit_echo_layers": 2,
        "baseline_two_qubit_echo_layers": 1,
    },
    "low": {
        "preserving_single_qubit_echo_layers": 2,
```

```
        "preserving_two_qubit_echo_layers": 1,
        "baseline_single_qubit_echo_layers": 3,
        "baseline_two_qubit_echo_layers": 2,
    },
}


MAGIC_SQUARE = [
    ["XI", "XX", "IX"],
    ["-XZ", "YY", "-ZX"],
    ["IZ", "ZZ", "ZI"],
]

ROW_CONTEXTS = {
    "R1": MAGIC_SQUARE[0],
    "R2": MAGIC_SQUARE[1],
    "R3": MAGIC_SQUARE[2],
}

COLUMN_CONTEXTS = {
    "C1": [MAGIC_SQUARE[row][0] for row in range(3)],
    "C2": [MAGIC_SQUARE[row][1] for row in range(3)],
    "C3": [MAGIC_SQUARE[row][2] for row in range(3)],
}

QUERY_CONFIGS = {
    f"R{row + 1}C{col + 1}": (row, col)
    for row in range(3)
    for col in range(3)
}

NONCONTEXTUAL_WIN_BOUND = 8.0 / 9.0
IDEAL_QUANTUM_WIN_RATE = 1.0
```

```
NONCONTEXTUAL_PM_BOUND = 4.0
IDEAL_PM_VALUE = 6.0

DEFAULT_PRESERVATION_LEVEL_ORDER = ["low", "medium", "high"]
DEFAULT_CIRCUIT_TYPES = ["contextuality-low",
"contextuality-medium", "contextuality-high", "baseline"]
DEFAULT_MITIGATION_MODES = [False, True]
def set_seeds(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)


@lru_cache(maxsize=1)
def ensure_runtime_service():
    try:
        from qiskit_ibm_runtime import QiskitRuntimeService
    except Exception as exc:
        raise RuntimeError(
            "qiskit-ibm-runtime is not available.
Install/configure it or disable "
            "real-backend options."
        ) from exc
    token = os.getenv("IBM_QUANTUM_TOKEN")
    instance = os.getenv("IBM_QUANTUM_INSTANCE")
    channel = os.getenv("IBM_QUANTUM_CHANNEL",
"ibm_quantum_platform")

    kwargs = {"channel": channel}
    if token:
        kwargs["token"] = token
    if instance:
        kwargs["instance"] = instance
    return QiskitRuntimeService(**kwargs)
```

```
@lru_cache(maxsize=4)
def get_reference_backend(backend_name: Optional[str] = None):
    service = ensure_runtime_service()
    target_backend = backend_name or IBM_BACKEND_NAME
    if target_backend == "auto":
        candidates = []
        for backend in service.backends(simulator=False):
            try:
                status = backend.status()
            except Exception:
                continue
            if not getattr(status, "operational", False):
                continue
            candidates.append((status.pending_jobs,
backend.name, backend))
        if not candidates:
            raise RuntimeError("No operational IBM hardware
backends are available for this account.")
        candidates.sort(key=lambda item: (item[0], item[1]))
        pending_jobs, target_backend, backend = candidates[0]
        print(f"[backend] auto-selected {target_backend}
(pending_jobs={pending_jobs})", flush=True)
        return backend
    backend = service.backend(target_backend)
    try:
        status = backend.status()
        print(f"[backend] selected {backend.name}
(pending_jobs={status.pending_jobs},
status={status.status_msg})", flush=True)
    except Exception:
```

```
        print(f"[backend] selected {backend.name}",
flush=True)
    return backend


def build_local_noise_model() -> NoiseModel:
    params = LOCAL_SIMULATION_NOISE
    noise_model = Noise Model()

    err_1q = depolarizing_error(params["single_qubit_depol"],
1)
    err_2q = depolarizing_error(params["two_qubit_depol"], 2)
    ro_err = ReadoutError([
        [1 - params["readout_p01"], params["readout_p01"]],
        [params["readout_p10"], 1 - params["readout_p10"]],
    ])

    one_qubit_gates = ["id", "rz", "sx", "x", "u", "u1", "u2",
"u3", "h", "s", "sdg"]
    two_qubit_gates = ["cx", "ecr", "cz"]

    for g in one_qubit_gates:
        try:
            noise_model.add_all_qubit_quantum_error(err_1q,
[g])
        except Exception:
            pass
    for g in two_qubit_gates:
        try:
            noise_model.add_all_qubit_quantum_error(err_2q,
[g])
        except Exception:
            pass
```

```
    noise_model.add_all_qubit_readout_error(ro_err)
    return noise_model


def build_simulator():

    if USE_BACKEND_MIMIC:
        backend = get_reference_backend(IBM_BACKEND_NAME)
        sim = AerSimulator.from_backend(backend)
        return sim, backend
    else:
        sim = AerSimulator(
            noise_model=build_local_noise_model(),
            seed_simulator=SEED,
        )
        return sim, None


def run_compiled_circuits(backend, compiled_circuits:
List[QuantumCircuit], shots: int) -> List[Dict[str, int]]:

    job = backend.run(compiled_circuits, shots=shots)
    result = job.result()
    return [result.get_counts(i) for i in
range(len(compiled_circuits))]


def run_sampler_circuits(backend, compiled_circuits:
List[QuantumCircuit], shots: int) -> List[Dict[str, int]]:

    from qiskit_ibm_runtime import SamplerV2
```

```
    sampler = SamplerV2(
        mode=backend,
        options={
            "default_shots": shots,
            "dynamical_decoupling": {
                "enable": True,
                "sequence_type": "XpXm",
            },
            "twirling": {
                "enable_gates": True,
            },
        },
    )
    job = sampler.run(compiled_circuits, shots=shots)
    print(f"[hardware] sampler job submitted: {job.job_id()}",
flush=True)
    result = job.result()

    counts_list: List[Dict[str, int]] = []
    for pub_result in result:
        data_values = list(pub_result.data.values())
        if not data_values:
            raise RuntimeError("Sampler result did not contain
classical measurement data.")
        counts_list.append(data_values[0].get_counts())
    return counts_list


def pauli_basis_rotation(qc: Quantum Circuit, qubit: int,
pauli: str) -> None:

    if pauli == "I":
        return
```

```
    if pauli == "X":
        qc.h(qubit)
    elif pauli == "Y":
        qc.sdg(qubit)
        qc.h(qubit)
    elif pauli == "Z":
        return
    else:
        raise ValueError(f"Unsupported pauli: {pauli}")


def bit_to_eigenvalue(bit: str) -> int:
    return +1 if bit == "0" else -1


def counts_to_probs(counts: Dict[str, int], shots: int) ->
Dict[str, float]:
    return {k: v / shots for k, v in counts.items()}


def l1_trace_distance(
    p: Dict[str, float],
    q: Dict[str, float],
    keys: Iterable[str],
) -> float:
    return 0.5 * sum(abs(p.get(k, 0.0) - q.get(k, 0.0)) for k
in keys)


def classical_fidelity(
    p: Dict[str, float],
    q: Dict[str, float],
    keys: Iterable[str],
```

```
) -> float:
    return (sum(math.sqrt(max(p.get(k, 0.0), 0.0) *
max(q.get(k, 0.0), 0.0)) for k in keys)) ** 2


def richardson_extrapolate(x_vals: List[float], y_vals:
List[float]) -> float:

    coeffs = np.polyfit(x_vals, y_vals, deg=min(2, len(x_vals)
- 1))
    poly = np.poly1d(coeffs)
    y0 = float(poly(0.0))
    return y0




def build_base_state_circuit() -> QuantumCircuit:

    qc = Quantum Circuit(4, 4)
    qc.h(0)
    qc.cx(0, 2)
    qc.h(1)
    qc.cx(1, 3)
    return qc


def apply_identity_sensitive_overhead(
    qc: Quantum Circuit,
    single_qubit_echo_layers: int,
    two_qubit_echo_layers: int,
) -> None:
    for _ in range(single_qubit_echo_layers):
```

```
        qc.barrier()
        for qubit in range(4):
            qc.s(qubit)
            qc.sdg(qubit)
        qc.h(0)
        qc.h(0)
        qc.h(1)
        qc.h(1)
        qc.x(2)
        qc.x(2)
        qc.x(3)
        qc.x(3)

    for _ in range(two_qubit_echo_layers):
        qc.barrier()
        qc.cx(0, 2)
        qc.cx(0, 2)
        qc.cx(1, 3)
        qc.cx(1, 3)


def build_contextuality_preserving_prep(preservation_level:
str = "high") -> QuantumCircuit:

    qc = build_base_state_circuit()
    profile = PRESERVATION_LEVELS[preservation_level]
    apply_identity_sensitive_overhead(
        qc,

single_qubit_echo_layers=profile["preserving_single_qubit_echo
_layers"],
```

```
two_qubit_echo_layers=profile["preserving_two_qubit_echo_layer
s"],
    )
    return qc


def build_baseline_prep(preservation_level: str = "high") ->
QuantumCircuit:

    qc = build_base_state_circuit()
    profile = PRESERVATION_LEVELS[preservation_level]

    qc.barrier()
    qc.s(0)
    qc.sdg(0)
    qc.h(1)
    qc.h(1)
    qc.x(2)
    qc.x(2)
    qc.s(3)
    qc.sdg(3)
    qc.cx(0, 2)
    qc.cx(0, 2)
    qc.cx(1, 3)
    qc.cx(1, 3)
    qc.barrier()
    apply_identity_sensitive_overhead(
        qc,

single_qubit_echo_layers=profile["baseline_single_qubit_echo_l
ayers"],
```

```
two_qubit_echo_layers=profile["baseline_two_qubit_echo_layers"
],
    )

    return qc


def build_prep_for_circuit_type(circuit_type: str) ->
QuantumCircuit:
    if circuit_type == "baseline":
        return build_baseline_prep("low")
    if circuit_type == "contextuality-low":
        return build_contextuality_preserving_prep("low")
    if circuit_type == "contextuality-medium":
        return build_contextuality_preserving_prep("medium")
    if circuit_type == "contextuality-high":
        return build_contextuality_preserving_prep("high")
    raise ValueError(f"Unsupported circuit type:
{circuit_type}")


def signed_pauli_matrix(label: str) -> np.ndarray:
    sign = -1 if label.startswith("-") else 1
    pauli_label = label[1:] if label.startswith("-") else
label
    return sign * Pauli(pauli_label).to_matrix()


@lru_cache(maxsize=None)
def get_local_measurement_spec(observables: Tuple[str, str,
str]) -> Dict[str, object]:
```

```
    operators = [signed_pauli_matrix(obs) for obs in
observables]
    combo = sum((i + 1) * op for i, op in
enumerate(operators))
    _, eigenvectors = np.linalg.eigh(combo)

    eigen_data: List[Tuple[Tuple[int, int, int], np.ndarray]]
= []
    for idx in range(eigenvectors.shape[1]):
        vec = eigenvectors[:, idx]
        observable_eigenvalues = []
        for op in operators:
            ev = np.vdot(vec, op @ vec)
            ev = np.real_if_close(ev)

observable_eigenvalues.append(int(round(float(np.real(ev)))))
        eigen_data.append((tuple(observable_eigenvalues),
vec))

    eigen_data.sort(key=lambda item: item[0])
    measurement_unitary = np.column_stack([vec for _, vec in
eigen_data])
    outcome_eigenvalues = {
        format(index, "02b"): eigenvalues
        for index, (eigenvalues, _) in enumerate(eigen_data)
    }

    return {
        "measurement_unitary": measurement_unitary,
        "measurement_unitary_dagger":
measurement_unitary.conj().T,
        "outcome_eigenvalues": outcome_eigenvalues,
        "outcome_labels": list(outcome_eigenvalues.keys()),
```

```
    }


def parse_query_name(query_name: str) -> Tuple[int, int]:
    try:
        return QUERY_CONFIGS[query_name]
    except KeyError as exc:
        raise ValueError(f"Unknown query configuration:
{query_name}") from exc


def get_row_measurement_spec(row_index: int) -> Dict[str,
object]:
    return
get_local_measurement_spec(tuple(MAGIC_SQUARE[row_index]))


def get_column_measurement_spec(col_index: int) -> Dict[str,
object]:
    return
get_local_measurement_spec(tuple(MAGIC_SQUARE[row][col_index]
for row in range(3)))


def build_query_circuit(
    prep: QuantumCircuit,
    query_name: str,
    include_measurements: bool,
) -> Quantum Circuit:
    row_index, col_index = parse_query_name(query_name)
    qc = prep.copy()
    qc.name = f"{prep.name}_{query_name}" if prep.name else
query_name
```

```
    row_spec = get_row_measurement_spec(row_index)
    col_spec = get_column_measurement_spec(col_index)

    qc.unitary(row_spec["measurement_unitary_dagger"], [0, 1],
label=f"row_{row_index + 1}_meas")
    qc.unitary(col_spec["measurement_unitary_dagger"], [2, 3],
label=f"col_{col_index + 1}_meas")

    if include_measurements:
        for qubit in range(4):
            qc.measure(qubit, qubit)
    return qc


def fold_global(circuit: QuantumCircuit, scale_factor: int) ->
QuantumCircuit:

    if scale_factor < 1 or scale_factor % 2 == 0:
        raise ValueError("scale_factor must be an odd positive
integer")

    if scale_factor == 1:
        return circuit.copy()

    k = (scale_factor - 1) // 2
    folded = QuantumCircuit(circuit.num_qubits,
circuit.num_clbits)
    folded.compose(circuit, inplace=True)

    unitary_part =
circuit.remove_final_measurements(inplace=False)
    for _ in range(k):
```

```
        folded.compose(unitary_part.inverse(), inplace=True)
        folded.compose(unitary_part, inplace=True)

    if circuit.num_clbits > 0:
        meas_circ = Quantum Circuit(circuit.num_qubits,
circuit.num_clbits)
        for instruction in circuit.data:
            if instruction.operation.name == "measure":
                meas_circ.append(
                    instruction.operation,
                    instruction.qubits,
                    instruction.clbits,
                )
        folded.compose(meas_circ, inplace=True)

    return folded



def decode_query_outcome(
    query_name: str,
    bitstring: str,
) -> Tuple[Tuple[int, int, int], Tuple[int, int, int]]:
    """
    Decode a 4-bit outcome into Alice's row outputs and Bob's
column outputs.
    """
    bits = bitstring.replace(" ", "")
    bits = bits[-4:].zfill(4)
    row_index, col_index = parse_query_name(query_name)
    row_bits = bits[-2:]   # c1c0
    col_bits = bits[:2]    # c3c2
```

```
    row_values =
get_row_measurement_spec(row_index)["outcome_eigenvalues"][row
_bits]
    col_values =
get_column_measurement_spec(col_index)["outcome_eigenvalues"][
col_bits]
    return row_values, col_values


def query_win_from_bitstring(query_name: str, bitstring: str)
-> int:
    row_index, col_index = parse_query_name(query_name)
    row_values, col_values = decode_query_outcome(query_name,
bitstring)
    row_ok = np.prod(row_values) == +1
    col_ok = np.prod(col_values) == -1
    intersection_ok = row_values[col_index] ==
col_values[row_index]
    return int(row_ok and col_ok and intersection_ok)


def outcome_distribution_from_counts(
    counts: Dict[str, int],
    shots: int,
) -> Dict[str, float]:

    dist: Dict[str, float] = {}
    for bitstring, c in counts.items():
        bits = bitstring.replace(" ", "")[-4:].zfill(4)
        dist[bits] = dist.get(bits, 0.0) + (c / shots)
    for label in [format(index, "04b") for index in
range(16)]:
        dist.setdefault(label, 0.0)
```

```
    return dist


def win_distribution_from_outcome_distribution(
    query_name: str,
    outcome_dist: Dict[str, float],
) -> Dict[str, float]:
    win = 0.0
    lose = 0.0
    for bitstring, prob in outcome_dist.items():
        if query_win_from_bitstring(query_name, bitstring) ==
1:
            win += prob
        else:
            lose += prob
    return {"win": win, "lose": lose}


def ideal_outcome_distribution(
    prep_circuit: Quantum Circuit,
    query_name: str,
) -> Dict[str, float]:

    ideal_query = build_query_circuit(prep_circuit,
query_name, include_measurements=False)
    state = Statevector.from_instruction(ideal_query)
    probs = state.probabilities_dict()
    ideal_dist = {format(index, "04b"): 0.0 for index in
range(16)}
    for bitstring, prob in probs.items():
        bits = bitstring.replace(" ", "")[-4:].zfill(4)
        ideal_dist[bits] += float(prob)
    return ideal_dist
```

```
def witness_from_expectations(context_expectations: Dict[str,
float]) -> float:

    return float(np.mean(list(context_expectations.values())))


def normalized_contextuality_score(witness: float) -> float:

    return float(
        np.clip(
            (witness - NONCONTEXTUAL_WIN_BOUND) /
(IDEAL_QUANTUM_WIN_RATE - NONCONTEXTUAL_WIN_BOUND),
            0.0,
            1.0,
        )
    )


def normalized_pm_contextuality(witness: float) -> float:
    return float(
        np.clip(
            (witness - NONCONTEXTUAL_PM_BOUND) /
(IDEAL_PM_VALUE - NONCONTEXTUAL_PM_BOUND),
            0.0,
            1.0,
        )
    )


def row_col_product_expectations_for_query(
    query_name: str,
```

```
    outcome_dist: Dict[str, float],
) -> Tuple[float, float]:
    row_exp = 0.0
    col_exp = 0.0
    for bitstring, prob in outcome_dist.items():
        row_values, col_values =
decode_query_outcome(query_name, bitstring)
        row_exp += prob * float(np.prod(row_values))
        col_exp += prob * float(np.prod(col_values))
    return row_exp, col_exp


def pm_witness_from_contexts(
    context_dists: Dict[str, Dict[str, float]],
) -> Tuple[float, float, Dict[int, float], Dict[int, float]]:
    row_exp: Dict[int, List[float]] = {0: [], 1: [], 2: []}
    col_exp: Dict[int, List[float]] = {0: [], 1: [], 2: []}
    for ctx, dist in context_dists.items():
        row_index, col_index = parse_query_name(ctx)
        row_val, col_val =
row_col_product_expectations_for_query(ctx, dist)
        row_exp[row_index].append(row_val)
        col_exp[col_index].append(col_val)

    row_means = {idx: float(np.mean(vals)) if vals else 0.0
for idx, vals in row_exp.items()}
    col_means = {idx: float(np.mean(vals)) if vals else 0.0
for idx, vals in col_exp.items()}
    witness = (row_means[0] + row_means[1] + row_means[2]) -
(col_means[0] + col_means[1] + col_means[2])
    return witness, normalized_pm_contextuality(witness),
row_means, col_means
```

```
@dataclass
class RunRecord:
    circuit_type: str
    mitigated: bool
    backend: str
    shots_per_context: int
    zne_scale_factors: str
    fidelity: float
    trace_distance: float
    contextuality_violation: float
    pm_contextuality: float
    pm_witness: float
    win_probability: float


@dataclass
class ContextRecord:
    circuit_type: str
    mitigated: bool
    query_name: str
    shots: int
    row_index: int
    column_index: int
    win_probability: float
    row_product_expectation: float
    column_product_expectation: float
    fidelity: float
    trace_distance: float
    success_probability: float
    failure_probability: float
    raw_scale_factor_dists: str
```

```
@dataclass
class ExperimentConfig:
    shots: int
    zne_scale_factors: List[int]
    circuit_types: List[str]
    mitigation_modes: List[bool]
    contexts: List[str]
    outdir: Path
    use_real_backend: bool
    use_backend_mimic: bool
    backend_name: str
    smoke_test: bool


def compile_circuit(
    qc: Quantum Circuit,
    backend_or_sim,
) -> Quantum Circuit:

    try:
        pm = generate_preset_pass_manager(
            optimization_level=OPTIMIZATION_LEVEL,
            backend=backend_or_sim,
        )
        compiled = pm.run(qc)
        return compiled
    except Exception:
        return transpile(qc, backend_or_sim,
optimization_level=OPTIMIZATION_LEVEL, seed_transpiler=SEED)


def run_single_context(
    prep_circuit: Quantum Circuit,
```

```
    circuit_type: str,
    context_name: str,
    simulator,
    shots: int,
    zne_scale_factors: Optional[List[int]] = None,
    mitigated: Optional[bool] = None,
) -> Tuple[Dict[str, float], Dict[int, Dict[str, float]]]:

    base_meas = build_query_circuit(prep_circuit,
context_name, include_measurements=True)
    raw_outcome_dists: Dict[int, Dict[str, float]] = {}
    progress_prefix = (
        f"[run] circuit={circuit_type} "
        f"mitigated={mitigated if mitigated is not None else
'n/a'} query={context_name}"
    )

    if not zne_scale_factors:
        print(progress_prefix, flush=True)
        compiled = compile_circuit(base_meas, simulator)
        result = simulator.run(compiled, shots=shots).result()
        counts = result.get_counts()
        dist = outcome_distribution_from_counts(counts, shots)
        return dist, raw_outcome_dists

    outcome_labels = tuple(format(index, "04b") for index in
range(16))
    outcome_probs_by_label = {label: [] for label in
outcome_labels}

    for sf in zne_scale_factors:
        print(f"{progress_prefix} scale_factor={sf}",
flush=True)
```

```
        folded = fold_global(base_meas, sf)
        compiled = compile_circuit(folded, simulator)
        result = simulator.run(compiled, shots=shots).result()
        counts = result.get_counts()
        dist = outcome_distribution_from_counts(counts, shots)
        raw_outcome_dists[sf] = dist
        for label in outcome_labels:

outcome_probs_by_label[label].append(dist.get(label, 0.0))

    extrapolated = {
        label: max(0.0,
richardson_extrapolate(zne_scale_factors, probs))
        for label, probs in outcome_probs_by_label.items()
    }
    s = sum(extrapolated.values())
    if s <= 0:
        dist0 = {label: 0.25 for label in outcome_labels}
    else:
        dist0 = {label: value / s for label, value in
extrapolated.items()}

    return dist0, raw_outcome_dists


def aggregate_metrics(
    prep_circuit: Quantum Circuit,
    context_dists: Dict[str, Dict[str, float]],
) -> Tuple[float, float, float, float, float, float]:

    context_expectations = {}
    fidelities = []
    trace_distances = []
```

```
    for ctx, dist in context_dists.items():
        ideal = ideal_outcome_distribution(prep_circuit, ctx)
        keys = [format(index, "04b") for index in range(16)]
        fidelities.append(classical_fidelity(dist, ideal,
keys))
        trace_distances.append(l1_trace_distance(dist, ideal,
keys))
        win_dist =
win_distribution_from_outcome_distribution(ctx, dist)
        context_expectations[ctx] = win_dist["win"]

    witness = witness_from_expectations(context_expectations)
    contextuality = normalized_contextuality_score(witness)
    pm_witness, pm_contextuality, _, _ =
pm_witness_from_contexts(context_dists)

    return (
        float(np.mean(fidelities)),
        float(np.mean(trace_distances)),
        contextuality,
        witness,
        pm_contextuality,
        pm_witness,
    )


def build_context_records(
    circuit_type: str,
    mitigated: bool,
    shots: int,
    context_dists: Dict[str, Dict[str, float]],
```

```
    raw_context_scale_dists: Dict[str, Dict[int, Dict[str,
float]]],
    prep_circuit: Quantum Circuit,
) -> List[ContextRecord]:
    records: List[ContextRecord] = []
    for ctx, dist in context_dists.items():
        ideal = ideal_outcome_distribution(prep_circuit, ctx)
        win_dist =
win_distribution_from_outcome_distribution(ctx, dist)
        row_index, col_index = parse_query_name(ctx)
        row_exp, col_exp =
row_col_product_expectations_for_query(ctx, dist)
        records.append(
            ContextRecord(
                circuit_type=circuit_type,
                mitigated=mitigated,
                query_name=ctx,
                shots=shots,
                row_index=row_index + 1,
                column_index=col_index + 1,
                win_probability=win_dist["win"],
                row_product_expectation=row_exp,
                column_product_expectation=col_exp,
                fidelity=classical_fidelity(dist, ideal,
[format(index, "04b") for index in range(16)]),
                trace_distance=l1_trace_distance(dist, ideal,
[format(index, "04b") for index in range(16)]),
                success_probability=win_dist["win"],
                failure_probability=win_dist["lose"],

raw_scale_factor_dists=json.dumps(raw_context_scale_dists.get(
ctx, {}), sort_keys=True),
            )
```

```
        )
    return records


def run_family(
    config: ExperimentConfig,
    circuit_type: str,
    prep_circuit: Quantum Circuit,
    mitigated: bool,
) -> Tuple[RunRecord, List[ContextRecord]]:

    simulator = None
    ref_backend = None
    backend_name = config.backend_name if
config.use_backend_mimic else "local_aer_noise_model"
    if not config.use_real_backend:
        simulator, ref_backend = build_simulator()
    raw_context_scale_dists: Dict[str, Dict[int, Dict[str,
float]]] = {}

    if config.use_real_backend:
        backend = get_reference_backend(config.backend_name)
        backend_name = backend.name
        context_dists = {}

        if mitigated:
            folded_jobs: List[Tuple[str, int, QuantumCircuit]]
= []
            for ctx in config.contexts:
                base_meas = build_query_circuit(prep_circuit,
ctx, include_measurements=True)
                raw_context_scale_dists[ctx] = {}
                for sf in config.zne_scale_factors:
```

```
                    print(
                        f"[run] circuit={circuit_type}
mitigated={mitigated} query={ctx} scale_factor={sf}",
                        flush=True,
                    )
                    folded = fold_global(base_meas, sf)
                    compiled = compile_circuit(folded,
backend)
                    folded_jobs.append((ctx, sf, compiled))

            counts_list = run_sampler_circuits(
                backend,
                [compiled for _, _, compiled in folded_jobs],
                shots=config.shots,
            )
            for (ctx, sf, _), counts in zip(folded_jobs,
counts_list):
                raw_context_scale_dists[ctx][sf] =
outcome_distribution_from_counts(counts, config.shots)

            outcome_labels = tuple(format(index, "04b") for
index in range(16))
            for ctx in config.contexts:
                extrapolated = {}
                for label in outcome_labels:
                    extrapolated[label] = max(
                        0.0,
                        richardson_extrapolate(
                            config.zne_scale_factors,

[raw_context_scale_dists[ctx][sf].get(label, 0.0) for sf in
config.zne_scale_factors],
                        ),
```

```
                    )
                s = sum(extrapolated.values())
                if s <= 0:
                    context_dists[ctx] = {label: 0.25 for
label in outcome_labels}
                else:
                    context_dists[ctx] = {label: value / s for
label, value in extrapolated.items()}
        else:
            compiled_jobs: List[Tuple[str, QuantumCircuit]] =
[]
            for ctx in config.contexts:
                print(
                    f"[run] circuit={circuit_type}
mitigated={mitigated} query={ctx}",
                    flush=True,
                )
                base_meas = build_query_circuit(prep_circuit,
ctx, include_measurements=True)
                compiled_jobs.append((ctx,
compile_circuit(base_meas, backend)))

            counts_list = run_sampler_circuits(
                backend,
                [compiled for _, compiled in compiled_jobs],
                shots=config.shots,
            )
            for (ctx, _), counts in zip(compiled_jobs,
counts_list):
                context_dists[ctx] =
outcome_distribution_from_counts(counts, config.shots)
    else:
        context_dists = {}
```

```
        for ctx in config.contexts:
            dist, raw_dists = run_single_context(
                prep_circuit=prep_circuit,
                circuit_type=circuit_type,
                context_name=ctx,
                simulator=simulator,
                shots=config.shots,
                zne_scale_factors=config.zne_scale_factors if
mitigated else None,
                mitigated=mitigated,
            )
            context_dists[ctx] = dist
            raw_context_scale_dists[ctx] = raw_dists

    fidelity, trace_distance, contextuality, win_probability,
pm_contextuality, pm_witness = aggregate_metrics(
        prep_circuit, context_dists
    )

    run_record = RunRecord(
        circuit_type=circuit_type,
        mitigated=mitigated,
        backend=backend_name,
        shots_per_context=config.shots,
        zne_scale_factors=json.dumps(config.zne_scale_factors
if mitigated else [1]),
        fidelity=fidelity,
        trace_distance=trace_distance,
        contextuality_violation=contextuality,
        pm_contextuality=pm_contextuality,
        pm_witness=pm_witness,
        win_probability=win_probability,
    )
```

```
    context_records = build_context_records(
        circuit_type=circuit_type,
        mitigated=mitigated,
        shots=config.shots,
        context_dists=context_dists,
        raw_context_scale_dists=raw_context_scale_dists,
        prep_circuit=prep_circuit,
    )
    return run_record, context_records


def save_dataframe(df: pd.DataFrame, path: Path) -> None:
    df.to_csv(path, index=False)


def save_excel_workbook(
    outpath: Path,
    summary_df: pd.DataFrame,
    query_df: pd.DataFrame,
    summary_tables: Dict[str, pd.DataFrame],
    manifest: Dict,
) -> Tuple[bool, str]:

    manifest_df = pd.DataFrame(
        [{"key": key, "value": json.dumps(value) if
isinstance(value, (dict, list)) else value} for key, value in
manifest.items()]
    )

    engine = None
    for candidate in ("openpyxl", "xlsxwriter"):
        try:
            __import__(candidate)
```

```
            engine = candidate
            break
        except Exception:
            continue

    if engine is None:
        return False, "Excel export skipped: install
`openpyxl` or `xlsxwriter` in the active environment."

    with pd.ExcelWriter(outpath, engine=engine) as writer:
        summary_df.to_excel(writer, sheet_name="summary",
index=False)
        query_df.to_excel(writer, sheet_name="queries",
index=False)
        manifest_df.to_excel(writer, sheet_name="manifest",
index=False)
        summary_tables["paper_summary"].to_excel(writer,
sheet_name="paper_summary", index=False)
        summary_tables["fidelity_table"].to_excel(writer,
sheet_name="fidelity_by_circuit")
        summary_tables["win_rate_table"].to_excel(writer,
sheet_name="win_rate_by_circuit")
        summary_tables["contextuality_table"].to_excel(writer,
sheet_name="contextuality_by_circuit")

summary_tables["preserving_advantage"].to_excel(writer,
sheet_name="preserving_advantage", index=False)
        summary_tables["mitigation_gain"].to_excel(writer,
sheet_name="mitigation_gain", index=False)
        summary_tables["weakest_queries"].to_excel(writer,
sheet_name="weakest_queries", index=False)
    return True, f"Saved Excel workbook with engine
`{engine}`."
```

```
def save_json(payload: Dict, path: Path) -> None:
    with path.open("w", encoding="utf-8") as f:
        json.dump(payload, f, indent=2, sort_keys=True)


def validate_probability_distributions(context_df:
pd.DataFrame) -> None:
    if context_df.empty:
        return
    sums = (context_df["success_probability"] +
context_df["failure_probability"]).round(9)
    if not np.allclose(sums.values, np.ones(len(sums)),
atol=1e-8):
        bad_rows = context_df.loc[~np.isclose(sums.values,
1.0, atol=1e-8)]
        raise ValueError(f"Non-normalized context
distributions detected:\n{bad_rows.to_string(index=False)}")


def ordered_circuit_types(values: Iterable[str]) -> List[str]:
    values = list(dict.fromkeys(values))
    preferred = [label for label in DEFAULT_CIRCUIT_TYPES if
label in values]
    extras = [label for label in values if label not in
preferred]
    return preferred + extras


def build_interpretable_summary_tables(
    summary_df: pd.DataFrame,
    query_df: pd.DataFrame,
```

```
) -> Dict[str, pd.DataFrame]:
    summary = summary_df.copy()
    summary["mitigation_label"] =
summary["mitigated"].map({False: "pre", True: "post"})

    paper_summary = (
        summary[
            [
                "circuit_type",
                "mitigated",
                "win_probability",
                "contextuality_violation",
                "pm_contextuality",
                "pm_witness",
                "fidelity",
                "trace_distance",
            ]
        ]
        .sort_values(["mitigated", "circuit_type"])
        .reset_index(drop=True)
    )

    fidelity_table = (
        summary.pivot_table(
            index="circuit_type",
            columns=["mitigation_label"],
            values="fidelity",
        )

.reindex(ordered_circuit_types(summary["circuit_type"]))
        .round(6)
    )
```

```
    win_rate_table = (
        summary.pivot_table(
            index="circuit_type",
            columns=["mitigation_label"],
            values="win_probability",
        )

.reindex(ordered_circuit_types(summary["circuit_type"]))
        .round(6)
    )

    contextuality_table = (
        summary.pivot_table(
            index="circuit_type",
            columns=["mitigation_label"],
            values="pm_contextuality",
        )

.reindex(ordered_circuit_types(summary["circuit_type"]))
        .round(6)
    )

    delta_rows = []
    for mitigated, sub in summary.groupby("mitigated"):
        baseline_row = sub[sub["circuit_type"] == "baseline"]
        if baseline_row.empty:
            continue
        baseline_metrics = baseline_row.iloc[0]
        for _, row in sub.iterrows():
            if row["circuit_type"] == "baseline":
                continue
            delta_rows.append(
                {
```

```
                    "circuit_type": row["circuit_type"],
                    "mitigated": mitigated,
                    "delta_win_probability":
row["win_probability"] - baseline_metrics["win_probability"],
                    "delta_contextuality":
row["pm_contextuality"] -
baseline_metrics["pm_contextuality"],
                    "delta_fidelity": row["fidelity"] -
baseline_metrics["fidelity"],
                    "delta_trace_distance":
row["trace_distance"] - baseline_metrics["trace_distance"],
                }
            )
    delta_table = pd.DataFrame(delta_rows).round(6)

    mitigation_gain_rows = []
    for circuit_type, sub in summary.groupby("circuit_type"):
        pre = sub[sub["mitigated"] == False]
        post = sub[sub["mitigated"] == True]
        if pre.empty or post.empty:
            continue
        mitigation_gain_rows.append(
            {
                "circuit_type": circuit_type,
                "gain_win_probability":
post["win_probability"].iloc[0] -
pre["win_probability"].iloc[0],
                "gain_contextuality":
post["pm_contextuality"].iloc[0] -
pre["pm_contextuality"].iloc[0],
                "gain_fidelity": post["fidelity"].iloc[0] -
pre["fidelity"].iloc[0],
```

```
                "gain_trace_distance":
post["trace_distance"].iloc[0] -
pre["trace_distance"].iloc[0],
            }
        )
    mitigation_gain_table =
pd.DataFrame(mitigation_gain_rows).round(6)

    query = query_df.copy()
    query["mitigation_label"] = query["mitigated"].map({False:
"pre", True: "post"})
    query_weakest = (
        query.sort_values(["circuit_type", "mitigated",
"win_probability", "trace_distance"], ascending=[True, True,
True, False])
        .groupby(["circuit_type", "mitigated"])
        .head(3)
        [
            [
                "circuit_type",
                "mitigated",
                "query_name",
                "row_index",
                "column_index",
                "win_probability",
                "fidelity",
                "trace_distance",
            ]
        ]
        .reset_index(drop=True)
        .round(6)
    )
```

```
    return {
        "paper_summary": paper_summary.round(6),
        "fidelity_table": fidelity_table,
        "win_rate_table": win_rate_table,
        "contextuality_table": contextuality_table,
        "preserving_advantage": delta_table,
        "mitigation_gain": mitigation_gain_table,
        "weakest_queries": query_weakest,
    }


def write_text_report(
    summary_df: pd.DataFrame,
    query_df: pd.DataFrame,
    outpath: Path,
) -> None:
    best = summary_df.sort_values(["win_probability",
"fidelity"], ascending=[False, False]).iloc[0]
    worst = summary_df.sort_values(["win_probability",
"fidelity"], ascending=[True, True]).iloc[0]
    pre = summary_df[summary_df["mitigated"] == False]
    post = summary_df[summary_df["mitigated"] == True]
    preserving_pre =
pre[pre["circuit_type"].str.startswith("contextuality-")]["win
_probability"].mean()
    baseline_pre = pre[pre["circuit_type"] ==
"baseline"]["win_probability"].mean()
    preserving_post =
post[post["circuit_type"].str.startswith("contextuality-")]["w
in_probability"].mean()
    baseline_post = post[post["circuit_type"] ==
"baseline"]["win_probability"].mean()
```

```
    weakest = query_df.sort_values(["win_probability",
"trace_distance"], ascending=[True, False]).head(5)

    lines = [
        "Mermin–Peres Magic-Square Experiment Report",
        "",
        "Headline findings",
        f"- Best condition: {best['circuit_type']} with
mitigated={best['mitigated']},
win_probability={best['win_probability']:.6f},
fidelity={best['fidelity']:.6f},
pm_contextuality={best['pm_contextuality']:.6f}",
        f"- Worst condition: {worst['circuit_type']} with
mitigated={worst['mitigated']},
win_probability={worst['win_probability']:.6f},
fidelity={worst['fidelity']:.6f},
pm_contextuality={worst['pm_contextuality']:.6f}",
        f"- Mean pre-mitigation win rate:
preserving={preserving_pre:.6f}, baseline={baseline_pre:.6f}",
        f"- Mean post-mitigation win rate:
preserving={preserving_post:.6f},
baseline={baseline_post:.6f}",
        "",
        "Weakest query pairs overall",
    ]
    for _, row in weakest.iterrows():
        lines.append(
            f"- {row['query_name']} ({row['circuit_type']},
mitigated={row['mitigated']}): "
            f"win_probability={row['win_probability']:.6f},
fidelity={row['fidelity']:.6f},
trace_distance={row['trace_distance']:.6f}"
        )
```

```
    outpath.write_text("\n".join(lines) + "\n",
encoding="utf-8")


def plot_fidelity_by_circuit(df: pd.DataFrame, outpath: Path)
-> None:
    circuit_order =
ordered_circuit_types(df["circuit_type"].unique())
    pivot = (
        df[df["mitigated"] == False]
        .set_index("circuit_type")
        .reindex(circuit_order)
    )

    plt.figure(figsize=(7, 5))
    plt.bar(pivot.index, pivot["fidelity"].values,
color="#4c78a8")
    plt.title("Figure 1. Mean Fidelity by Circuit Type")
    plt.xlabel("Circuit type")
    plt.ylabel("Mean fidelity")
    plt.ylim(0, 1.0)
    plt.tight_layout()
    plt.savefig(outpath, dpi=200)
    plt.close()


def plot_win_probability_by_circuit(df: pd.DataFrame, outpath:
Path) -> None:
    circuit_order =
ordered_circuit_types(df["circuit_type"].unique())
    pivot = df[df["mitigated"] ==
False].set_index("circuit_type").reindex(circuit_order)
    plt.figure(figsize=(7, 5))
```

```
    plt.bar(pivot.index, pivot["win_probability"].values,
color="#72b7b2")
    plt.axhline(NONCONTEXTUAL_WIN_BOUND, color="#444444",
linestyle="--", linewidth=1, label="classical bound")
    plt.title("Magic-Square Win Rate by Circuit Type")
    plt.xlabel("Circuit type")
    plt.ylabel("Mean win probability")
    plt.legend()
    plt.tight_layout()
    plt.savefig(outpath, dpi=200)
    plt.close()


def plot_contextuality_by_circuit(df: pd.DataFrame, outpath:
Path) -> None:
    circuit_order =
ordered_circuit_types(df["circuit_type"].unique())
    pivot = df[df["mitigated"] ==
False].set_index("circuit_type").reindex(circuit_order)
    plt.figure(figsize=(7, 5))
    plt.bar(pivot.index, pivot["pm_contextuality"].values,
color="#f58518")
    plt.title("Mean Contextuality Violation by Circuit Type")
    plt.xlabel("Circuit type")
    plt.ylabel("Normalized PM contextuality")
    plt.tight_layout()
    plt.savefig(outpath, dpi=200)
    plt.close()


def plot_query_heatmaps(query_df: pd.DataFrame, outdir: Path)
-> None:
    for metric, title, stem in [
```

```
        ("win_probability", "Per-Query Win Probability",
"figure_query_win_heatmap"),
        ("trace_distance", "Per-Query Trace Distance",
"figure_query_trace_distance_heatmap"),
    ]:
        for mitigated in [False, True]:
            subset = query_df[query_df["mitigated"] ==
mitigated]
            fig, axes = plt.subplots(1, 2, figsize=(10, 4),
constrained_layout=True)
            for ax, circuit_type in zip(axes,
["contextuality-preserving", "baseline"]):
                data = (
                    subset[subset["circuit_type"] ==
circuit_type]
                    .groupby(["row_index", "column_index"],
as_index=False)[metric]
                    .mean()
                    .pivot(index="row_index",
columns="column_index", values=metric)
                    .reindex(index=[1, 2, 3], columns=[1, 2,
3])
                )
                im = ax.imshow(data.values, cmap="viridis" if
metric == "win_probability" else "magma", aspect="equal")
                ax.set_title(circuit_type)
                ax.set_xlabel("Column query")
                ax.set_ylabel("Row query")
                ax.set_xticks([0, 1, 2], labels=["C1", "C2",
"C3"])
                ax.set_yticks([0, 1, 2], labels=["R1", "R2",
"R3"])
                for i in range(3):
```

```
                    for j in range(3):
                        ax.text(j, i, f"{data.values[i,
j]:.3f}", ha="center", va="center", color="white", fontsize=8)
            fig.suptitle(f"{title} ({'post-mitigation' if
mitigated else 'pre-mitigation'})")
            fig.colorbar(im, ax=axes.ravel().tolist(),
shrink=0.9)
            fig.savefig(outdir / f"{stem}_{'post' if mitigated
else 'pre'}.png", dpi=200)
            plt.close(fig)


def plot_contextuality_vs_fidelity(df: pd.DataFrame, outpath:
Path) -> None:
    plt.figure(figsize=(7, 5))
    for ct, sub in df.groupby("circuit_type"):
        plt.scatter(sub["pm_contextuality"], sub["fidelity"],
label=ct)
    plt.title("PM Contextuality vs Fidelity")
    plt.xlabel("Normalized PM contextuality")
    plt.ylabel("Fidelity")
    plt.legend()
    plt.tight_layout()
    plt.savefig(outpath, dpi=200)
    plt.close()


def plot_pre_post_mitigation(df: pd.DataFrame, outpath: Path)
-> None:
    grouped = (
        df.groupby(["circuit_type", "mitigated"],
as_index=False)["fidelity"]
        .mean()
```

```
        .sort_values(["circuit_type", "mitigated"])
    )

    circuit_types = list(grouped["circuit_type"].unique())
    pre = []
    post = []
    for ct in circuit_types:
        pre_val = grouped[(grouped["circuit_type"] == ct) &
(grouped["mitigated"] == False)]["fidelity"].mean()
        post_val = grouped[(grouped["circuit_type"] == ct) &
(grouped["mitigated"] == True)]["fidelity"].mean()
        pre.append(pre_val)
        post.append(post_val)

    x = np.arange(len(circuit_types))
    width = 0.35

    plt.figure(figsize=(7, 5))
    plt.bar(x - width / 2, pre, width=width,
label="Pre-mitigation")
    plt.bar(x + width / 2, post, width=width,
label="Post-mitigation")
    plt.xticks(x, circuit_types)
    plt.title("Mean Fidelity Pre vs Post Error Mitigation")
    plt.xlabel("Circuit type")
    plt.ylabel("Mean fidelity")
    plt.legend()
    plt.tight_layout()
    plt.savefig(outpath, dpi=200)
    plt.close()
```

```
def parse_bool_csv(raw: str) -> List[bool]:
    values = []
    for item in raw.split(","):
        normalized = item.strip().lower()
        if normalized in {"true", "t", "1", "yes"}:
            values.append(True)
        elif normalized in {"false", "f", "0", "no"}:
            values.append(False)
        elif normalized:
            raise ValueError(f"Unsupported mitigation mode
value: {item}")
    if not values:
        raise ValueError("At least one mitigation mode must be
provided.")
    return values


def parse_args() -> ExperimentConfig:
    parser = argparse.ArgumentParser(description="Run the
Mermin-Peres contextuality experiment.")
    parser.add_argument("--shots", type=int,
default=int(os.getenv("MP_SHOTS", DEFAULT_SHOTS)))
    parser.add_argument(
        "--zne-scale-factors",
        default=os.getenv(
            "MP_ZNE_SCALE_FACTORS",
            ",".join(str(v) for v in
DEFAULT_ZNE_SCALE_FACTORS),
        ),
    )
    parser.add_argument(
        "--circuit-types",
```

```
        default=os.getenv("MP_CIRCUIT_TYPES",
",".join(DEFAULT_CIRCUIT_TYPES)),
    )
    parser.add_argument(
        "--mitigation-modes",
        default=os.getenv("MP_MITIGATION_MODES",
"false,true"),
        help="Comma-separated booleans, e.g. false,true",
    )
    parser.add_argument(
        "--contexts",
        default=os.getenv("MP_CONTEXTS",
",".join(QUERY_CONFIGS.keys())),
    )
    parser.add_argument(
        "--outdir",
        default=os.getenv("MP_OUTDIR", str(OUTDIR)),
    )
    parser.add_argument("--smoke-test", action="store_true",
default=os.getenv("MP_SMOKE_TEST", "").lower() in {"1",
"true", "yes"})
    parser.add_argument("--use-real-backend",
action="store_true", default=USE_REAL_BACKEND)
    parser.add_argument("--use-backend-mimic",
action="store_true", default=USE_BACKEND_MIMIC)
    parser.add_argument("--backend-name",
default=os.getenv("MP_BACKEND_NAME", IBM_BACKEND_NAME))
    args = parser.parse_args()

    zne_scale_factors = [int(v.strip()) for v in
args.zne_scale_factors.split(",") if v.strip()]
    circuit_types = [v.strip() for v in
args.circuit_types.split(",") if v.strip()]
```

```
    contexts = [v.strip() for v in args.contexts.split(",") if
v.strip()]
    mitigation_modes = parse_bool_csv(args.mitigation_modes)

    if args.smoke_test:
        args.shots = min(args.shots, 256)
        zne_scale_factors = [1, 3]

    invalid_circuits = [v for v in circuit_types if v not in
DEFAULT_CIRCUIT_TYPES]
    invalid_contexts = [v for v in contexts if v not in
QUERY_CONFIGS]
    if invalid_circuits:
        raise ValueError(f"Unsupported circuit types:
{invalid_circuits}")
    if invalid_contexts:
        raise ValueError(f"Unsupported contexts:
{invalid_contexts}")
    if args.use_real_backend and args.use_backend_mimic:
        raise ValueError("Choose either --use-real-backend or
--use-backend-mimic, not both.")
    if any(sf < 1 or sf % 2 == 0 for sf in zne_scale_factors):
        raise ValueError("ZNE scale factors must be positive
odd integers.")

    outdir = Path(args.outdir)
    outdir.mkdir(parents=True, exist_ok=True)

    return ExperimentConfig(
        shots=args.shots,
        zne_scale_factors=zne_scale_factors,
        circuit_types=circuit_types,
        mitigation_modes=mitigation_modes,
```

```
        contexts=contexts,
        outdir=outdir,
        use_real_backend=args.use_real_backend,
        use_backend_mimic=args.use_backend_mimic,
        backend_name=args.backend_name,
        smoke_test=args.smoke_test,
    )


def main() -> None:
    set_seeds(SEED)
    config = parse_args()
    print(
        f"[startup] real_backend={config.use_real_backend}
backend_mimic={config.use_backend_mimic} "
        f"backend_name={config.backend_name}
shots={config.shots} smoke_test={config.smoke_test}",
        flush=True,
    )

    execution_manifest = {
        "seed": SEED,
        "shots": config.shots,
        "zne_scale_factors": config.zne_scale_factors,
        "local_simulation_noise": LOCAL_SIMULATION_NOISE,
        "circuit_types": config.circuit_types,
        "mitigation_modes": config.mitigation_modes,
        "contexts": config.contexts,
        "backend_name": config.backend_name,
        "use_real_backend": config.use_real_backend,
        "use_backend_mimic": config.use_backend_mimic,
        "smoke_test": config.smoke_test,
        "generated_at_unix": time.time(),
```

```
    }
    save_json(execution_manifest, config.outdir /
"run_manifest.json")

    records: List[RunRecord] = []
    context_records: List[ContextRecord] = []
    total_runs = len(config.mitigation_modes) *
len(config.circuit_types)
    current_run = 0
    for mitigated in config.mitigation_modes:
        for circuit_type in config.circuit_types:
            current_run += 1
            prep_circuit =
build_prep_for_circuit_type(circuit_type)
            prep_circuit.name = circuit_type
            print(
                f"[batch] {current_run}/{total_runs}
circuit={circuit_type} mitigated={mitigated}",
                flush=True,
            )
            record, per_context = run_family(
                config=config,
                circuit_type=circuit_type,
                prep_circuit=prep_circuit,
                mitigated=mitigated,
            )
            records.append(record)
            context_records.extend(per_context)

            partial_summary = pd.DataFrame([asdict(r) for r in
records])
            save_dataframe(partial_summary, config.outdir /
"summary_results.partial.csv")
```

```
            partial_context = pd.DataFrame([asdict(r) for r in
context_records])
            save_dataframe(partial_context, config.outdir /
"context_results.partial.csv")

    df = pd.DataFrame([asdict(r) for r in records])
    context_df = pd.DataFrame([asdict(r) for r in
context_records])
    validate_probability_distributions(context_df)

    save_dataframe(df, config.outdir / "summary_results.csv")
    save_dataframe(context_df, config.outdir /
"context_results.csv")
    save_dataframe(context_df, config.outdir /
"query_results.csv")

    # Save a rounded version too
    df_rounded = df.copy()
    for col in ["fidelity", "trace_distance",
"contextuality_violation", "win_probability"]:
        df_rounded[col] = df_rounded[col].round(6)
    save_dataframe(df_rounded, config.outdir /
"summary_results_rounded.csv")

    summary_tables = build_interpretable_summary_tables(df,
context_df)
    save_dataframe(summary_tables["paper_summary"],
config.outdir / "table_paper_summary.csv")
    summary_tables["fidelity_table"].to_csv(config.outdir /
"table_fidelity_by_circuit_type.csv")
    summary_tables["win_rate_table"].to_csv(config.outdir /
"table_win_rate_by_circuit_type.csv")
```

```
    summary_tables["contextuality_table"].to_csv(config.outdir
/ "table_contextuality_by_circuit_type.csv")
    summary_tables["fidelity_table"].to_csv(config.outdir /
"table_fidelity_by_noise.csv")
    summary_tables["win_rate_table"].to_csv(config.outdir /
"table_win_rate_by_noise.csv")
    summary_tables["contextuality_table"].to_csv(config.outdir
/ "table_contextuality_by_noise.csv")
    save_dataframe(summary_tables["preserving_advantage"],
config.outdir / "table_preserving_advantage.csv")
    save_dataframe(summary_tables["mitigation_gain"],
config.outdir / "table_mitigation_gain.csv")
    save_dataframe(summary_tables["weakest_queries"],
config.outdir / "table_weakest_queries.csv")
    write_text_report(df, context_df, config.outdir /
"results_report.txt")
    workbook_saved, workbook_message = save_excel_workbook(
        config.outdir / "results_workbook.xlsx",
        summary_df=df,
        query_df=context_df,
        summary_tables=summary_tables,
        manifest=execution_manifest,
    )

    plot_fidelity_by_circuit(df, config.outdir /
"figure_fidelity_by_circuit_type.png")
    plot_win_probability_by_circuit(df, config.outdir /
"figure_win_rate_by_circuit_type.png")
    plot_contextuality_by_circuit(df, config.outdir /
"figure_contextuality_by_circuit_type.png")
    plot_fidelity_by_circuit(df, config.outdir /
"figure_fidelity_vs_noise.png")
```

```
    plot_win_probability_by_circuit(df, config.outdir /
"figure_win_rate_vs_noise.png")
    plot_contextuality_by_circuit(df, config.outdir /
"figure_contextuality_vs_noise.png")
    plot_contextuality_vs_fidelity(df, config.outdir /
"figure_contextuality_vs_fidelity.png")
    if set(config.mitigation_modes) >= {False, True}:
        plot_pre_post_mitigation(df, config.outdir /
"figure_pre_post_mitigation.png")
        plot_query_heatmaps(context_df, config.outdir)

    print("\n=== Finished ===")
    print(df_rounded.to_string(index=False))
    print(workbook_message)
    print(f"\nSaved outputs to: {config.outdir.resolve()}")


if __name__ == "__main__":
    main()
```